

Evaluating the Resistance of ARADI Against Differential Fault Attack

Chandan Dey¹ , Soumya Sahoo²  and Santanu Sarkar² 

¹ R. C. Bose Centre for Cryptology and Security, Indian Statistical Institute, Kolkata, West Bengal, 700108, India
c.dey.math95@gmail.com

² Department of Mathematics, Indian Institute of Technology Madras, Chennai, Tamil Nadu, 600036, India
soumyasahoo078@gmail.com, sarkar.santanu.bir1@gmail.com

Abstract. The ARADI block cipher is developed by the U.S. National Security Agency (NSA) as part of its efforts to design secure and efficient cryptographic algorithms. In this paper, we present the first detailed analysis of the lightweight block cipher ARADI under differential fault attacks. Although ARADI is structured around word-wise operations, its security fundamentally depends on bit-level properties, making it vulnerable to carefully crafted fault models. Bit-based fault models require a large number of faults, while word-based models reduce the number of faults but make precise identification of fault locations difficult. To overcome these limitations, we introduce a new nibble-based approach using permissible nibble differences (PNDs). This model provides an effective tradeoff between the number of required faults and the complexity of identifying fault positions. In our attack, the adversary randomly injects nibble faults before the last two rounds and constructs differential equations involving unknown key variables without knowing the exact fault values. Our results demonstrate that, with at most 108 random fault injections, the entire master key of ARADI can be recovered within a practical time complexity. This work reveals previously unexplored vulnerabilities of ARADI under fault attacks and highlights the need to evaluate lightweight cipher designs using nibble-level fault models, along with the traditional bit- and word-level analyses.

Keywords: ARADI, Differential Fault Attack, Nibble-Based Fault Attack, Block Cipher.

1 Introduction

Fault analysis (FA), first proposed by Boneh *et al.* [7] for RSA-CRT, has become a powerful class of implementation attacks on cryptographic primitives. By deliberately inducing errors during computation and comparing correct and faulty outputs, an adversary can extract sensitive information and often recover secret keys. Biham and Shamir [5] extended these ideas to symmetric ciphers by introducing Differential Fault Analysis (DFA) for DES, inaugurating systematic fault-based cryptanalysis of block ciphers.

Since then, a variety of fault techniques and variants have been developed, including Statistical Ineffective Fault Analysis (SIFA) [9], Collision Fault Analysis [6], Fault Intensity Analysis [11], Persistent Fault Attacks [22], and other recent advances [16, 17, 18]. DFA in particular has been applied successfully to many block ciphers, such as AES [1], PRESENT [2], PRINCE [19], MIDORI [8] and SKINNY [21].

Practical fault injections are feasible on real devices using techniques such as voltage or clock glitching, laser-induced faults, electromagnetic (EM) injection, and rowhammer/low-hammer-style attacks; these methods differ in precision and repeatability but have been demonstrated to induce single-byte or nibble-scale corruptions in intermediate states. The availability of such techniques makes DFA a realistic threat to tamper-prone implementations like smart cards and embedded devices.

The ARADI [12] block cipher, developed by the U.S. National Security Agency (NSA), is specifically designed to provide high-speed, low-latency encryption for memory protection. It was introduced together with LLAMA, an authenticated encryption mode that ensures both confidentiality and integrity for computer memory. As modern processor architectures increasingly rely on secure enclaves to protect sensitive computations within the CPU, system memory (RAM) remains vulnerable to various physical attacks, most notably cold boot attacks, which exploit data remanence to extract cryptographic keys and other confidential data.

To mitigate these vulnerabilities, cryptographic memory protection mechanisms have been developed that encrypt memory contents and authenticate them against tampering. However, such schemes must achieve a careful trade-off between strong security and minimal performance degradation, as any additional delay in memory access can critically affect system performance. ARADI addresses this challenge by offering a cipher architecture optimized for extremely low latency while maintaining robust cryptographic strength. Its design makes it particularly well-suited for high-performance, secure computing environments that demand rapid memory access and strong resistance against emerging physical and side-channel threats.

MOTIVATION OF OUR ANALYSIS: ARADI, a lightweight block cipher designed for low-latency and secure memory encryption, requires a thorough security evaluation to assess its robustness against practical cryptanalytic attacks. Despite its significance, no fault analysis (FA) on ARADI has been reported so far, with only a recent side-channel attack [15] and a few classical cryptanalysis efforts [3, 4, 10] exploring its vulnerabilities. In this work, we present the first Differential Fault Analysis (DFA) on ARADI, employing a nibble-based fault model and demonstrating practical key-recovery attacks under realistic fault conditions.

1.1 Our Contributions

In this paper, we analyze the ARADI cipher and demonstrate a practical differential fault attack using a novel nibble-based fault model based on permissible nibble differences (PNDs). Unlike bit-level fault models, which require an impractically large number of faults, or word-level models, which make it difficult to precisely identify faulty positions, our approach provides a practical trade-off between the number of faults and attack complexity. In our strategy, the attacker injects random nibble faults at the beginning of round 15 into a chosen word, without knowing the exact nibble position or fault value, and observes the resulting ciphertext differences. Based on these observations and the known fault propagation behavior, a sufficient number of differential equations are derived, linking the differences to the unknown key bits. These equations are grouped into subkey sets, and we show that 27 equations are sufficient to reduce the keyspace of a 16-bit subkey from 2^{16} to 2^4 , while 45 equations yield a unique solution. Repeating this process for all 8 subkey groups allows the recovery of the full 128-bit last round subkey with at most 36 random permissible nibble differences (PNDs) injections. The 256-bit master key recovery required at most 108 random faults. The total time complexity of the attack is 3×2^{32} , indicating that the attack is feasible for the lightweight cipher ARADI within a practical timeframe. This work presents the first differential fault analysis of ARADI and provides a

realistic, low-overhead attack model with clear resource estimates, highlighting previously unexplored vulnerabilities of the cipher.

1.2 Organization of the Paper

The paper is organized as follows. [Section 2](#) provides a brief overview of differential fault analysis and a description of the ARADI cipher. [Section 3](#) presents our main contributions. In [Subsection 3.1](#), we describe the attack model and assumptions. [Subsection 3.2](#) explains the detection of fault injections, while [Subsection 3.3](#) analyzes the differential fault attack. [Subsection 3.4](#) details the actual attack procedure with an illustrative example, and [Subsection 3.5](#) discusses the master key recovery. [Subsection 3.6](#) presents the complexity analysis and simulation results, followed by [Subsection 3.7](#), which outlines potential countermeasures. Finally, [Section 4](#) concludes the paper. Notations used in this paper are introduced where needed.

2 Preliminaries

2.1 Differential Fault Analysis

Differential Fault Analysis (DFA) is a powerful class of side-channel attacks that combines ideas from differential cryptanalysis with deliberate fault injection. The core principle is to exploit the differences in the output of a cryptographic computation caused by intentionally induced errors. An attacker can extract secret information, such as keys or intermediate values, by carefully analyzing these deviations.

The effectiveness of a DFA attack depends heavily on the characteristics of the injected fault. In general, fault models are classified along two dimensions: the *magnitude of alteration* (how much data is corrupted) and the *control over fault placement* (whether the attacker can decide where the fault occurs). These categories can be described as follows:

Fault Magnitude

1. BIT FLIP: The attacker changes a single bit of a word.
2. NIBBLE ERROR: One nibble is corrupted and replaced by an arbitrary nibble.
3. BYTE ERROR: One byte is altered and replaced by an arbitrary value.
4. WORD ERROR: An entire word is corrupted, typically substituted with random data.

Positioning Control

1. CHOSEN POSITION: The attacker can accurately target the location of the injected fault.
2. RANDOM POSITION: The attacker has no control over where the fault occurs; errors appear unpredictably.

Smaller-scale fault injections (e.g., bit-level) typically provide the attacker with stronger assumptions and more helpful information. Similarly, having control over the exact position of the injected fault strengthens the attack model.

2.2 Detail Description of ARADI

In this section, we provide a compact description of the low-latency block cipher ARADI. The ARADI cipher is designed following the Substitution-Permutation Network (SPN) paradigm. It operates on 128-bit plaintext blocks and is parameterized with a 256-bit master key. The 128-bit internal state of ARADI can equivalently be represented as four

32-bit words (w, x, y, z) , which are arranged as a 4×32 binary matrix. Specifically, each word corresponds to one row of the state matrix:

$$S = \begin{bmatrix} w_{31} & w_{30} & w_{29} & \cdots & w_0 \\ x_{31} & x_{30} & x_{29} & \cdots & x_0 \\ y_{31} & y_{30} & y_{29} & \cdots & y_0 \\ z_{31} & z_{30} & z_{29} & \cdots & z_0 \end{bmatrix} = \begin{bmatrix} s_{127} & s_{126} & s_{125} & \cdots & s_{96} \\ s_{95} & s_{94} & s_{93} & \cdots & s_{64} \\ s_{63} & s_{62} & s_{61} & \cdots & s_{32} \\ s_{31} & s_{30} & s_{29} & \cdots & s_0 \end{bmatrix},$$

where $w_i, x_i, y_i, z_i \in \mathbb{F}_2$ denotes the i -th bit of the corresponding 32-bit word. In this representation, w corresponds to the most significant 32-bit word of the 128-bit state, while z denotes the least significant word. In the following, we describe the round function of ARADI.

Round Function of ARADI

The round function of ARADI consists of three primary operations: the substitution layer (S-box), the linear transformation layer, and the addition of the round key. We follow the notation introduced in the original specification paper [12]. Specifically, we denote:

- π : the non-linear substitution (S-box) layer,
- Λ_i : the linear transformation in the i -th round,
- τ_{k_i} : the XOR operation with the 128-bit round key k_i .

With these definitions, the i -th round function of ARADI is expressed as

$$R_i = \tau_{k_i} \circ \Lambda_i \circ \pi,$$

where $\circ$ denotes functional composition. To update the internal state, the round function applies in the following way $S^i = R_i(S^{i-1})$. An initial key addition, commonly referred to as *pre-whitening*, is applied before the first round and is denoted by τ_{k_0} . The cipher operates over a total of 16 rounds. Accordingly, the full encryption process of ARADI can be written as

$$(\tau_{k_{16}} \circ \Lambda_{16} \circ \pi) \circ \cdots \circ (\tau_{k_1} \circ \Lambda_1 \circ \pi) \circ \tau_{k_0}.$$

A schematic representation of the i -th round function is shown in Figure 1, where S^i and S^{i+1} denote the input and output states of round i , respectively. The input to the cipher (plaintext) is denoted by S^0 .

Nonlinear Layer (π): The nonlinear layer applies a fixed 4-bit substitution SB independently and in parallel to each of the 32 columns of the internal state. The lookup tables for SB and its inverse SB^{-1} are given in Table 1.

Table 1: The 4-bit S-box SB and its inverse SB^{-1} used in ARADI.

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$SB(x)$	0	1	2	3	4	D	F	6	8	B	5	E	C	7	A	9
$SB^{-1}(x)$	0	1	2	3	4	A	7	D	8	F	E	9	C	5	B	6

The nonlinear layer π of ARADI operates on the four 32-bit words (w, x, y, z) of the internal state. Instead of being defined via a lookup table, the transformation can be equivalently expressed as a set of word-level Boolean operations:

$$\begin{cases} x \leftarrow x \oplus (w \& y), \\ z \leftarrow z \oplus (x \& y), \\ y \leftarrow y \oplus (w \& z), \\ w \leftarrow w \oplus (x \& z), \end{cases}$$

where $\oplus$ denotes bitwise XOR and $\&$ denotes bitwise AND.

This formulation highlights that the SBox layer consists of a sequence of simple, word-parallel Boolean operations, enabling efficient hardware and software implementation. Thus, each 4-bit SBox is implicitly realized by these algebraic updates applied in parallel across all columns of the 4×32 state matrix.

Linear Layer (Λ): The linear layer updates each row of the internal state via a round-dependent linear transformation. In round $i \in \{1, \dots, 16\}$, the map Λ_i acts on the state S as

$$\Lambda_i(S) = (L_j(w), L_j(x), L_j(y), L_j(z)), \quad j = (i - 1) \bmod 4.$$

Each L_0, L_1, L_2, L_3 is an involutive linear mapping over $\mathbb{F}_2^{32}$. Writing a 32-bit row as the concatenation of two 16-bit halves, u (upper) and l (lower), we define

$$L_j(u \parallel l) = \left(u \oplus (u \lll a_j) \oplus (l \lll c_j) \parallel l \oplus (u \lll a_j) \oplus (l \lll b_j) \right),$$

where $\lll$ denotes a left rotation on the respective 16-bit half (i.e., rotations are taken modulo 16). The rotational offsets (a_j, b_j, c_j) are listed in Table 2.

Table 2: Rotation offsets of the linear layer in ARADI.

j	a_j	b_j	c_j
0	11	8	14
1	10	9	11
2	9	4	14
3	8	9	7

Key Scheduling Algorithm

The master key K of ARADI is 256 bits in length and is represented as an array of eight 32-bit words. We denote the key schedule state in round i as

$$K^i = (K_0^i, K_1^i, \dots, K_7^i).$$

Initially, the key state is loaded directly from the master key:

$$K^0 = K = (K_0^0, K_1^0, \dots, K_7^0).$$

The 128-bit pre-whitening key k_0 and the round keys k_i for $i = 1, 2, \dots, 16$ are derived by concatenating four consecutive words from the key state:

$$k_i = \begin{cases} K_0^i \parallel K_1^i \parallel K_2^i \parallel K_3^i, & \text{if } i \bmod 2 = 0, \\ K_4^i \parallel K_5^i \parallel K_6^i \parallel K_7^i, & \text{if } i \bmod 2 = 1. \end{cases}$$

At each step i , the key state K^i is updated through the following operations:

1. **Linear Transformations:** Two invertible linear maps, M_0 and M_1 , are applied to pairs of 32-bit words. They are defined as

$$M_0(x, y) = ((x \lll 1) \oplus y, (y \lll 3) \oplus (x \lll 1) \oplus y),$$

$$M_1(x, y) = ((x \lll 9) \oplus y, (y \lll 28) \oplus (x \lll 9) \oplus y),$$

where $\lll$ denotes left rotation. The map M_0 is applied to the pairs (K_0^i, K_1^i) and (K_4^i, K_5^i) , while M_1 is applied to (K_2^i, K_3^i) and (K_6^i, K_7^i) .

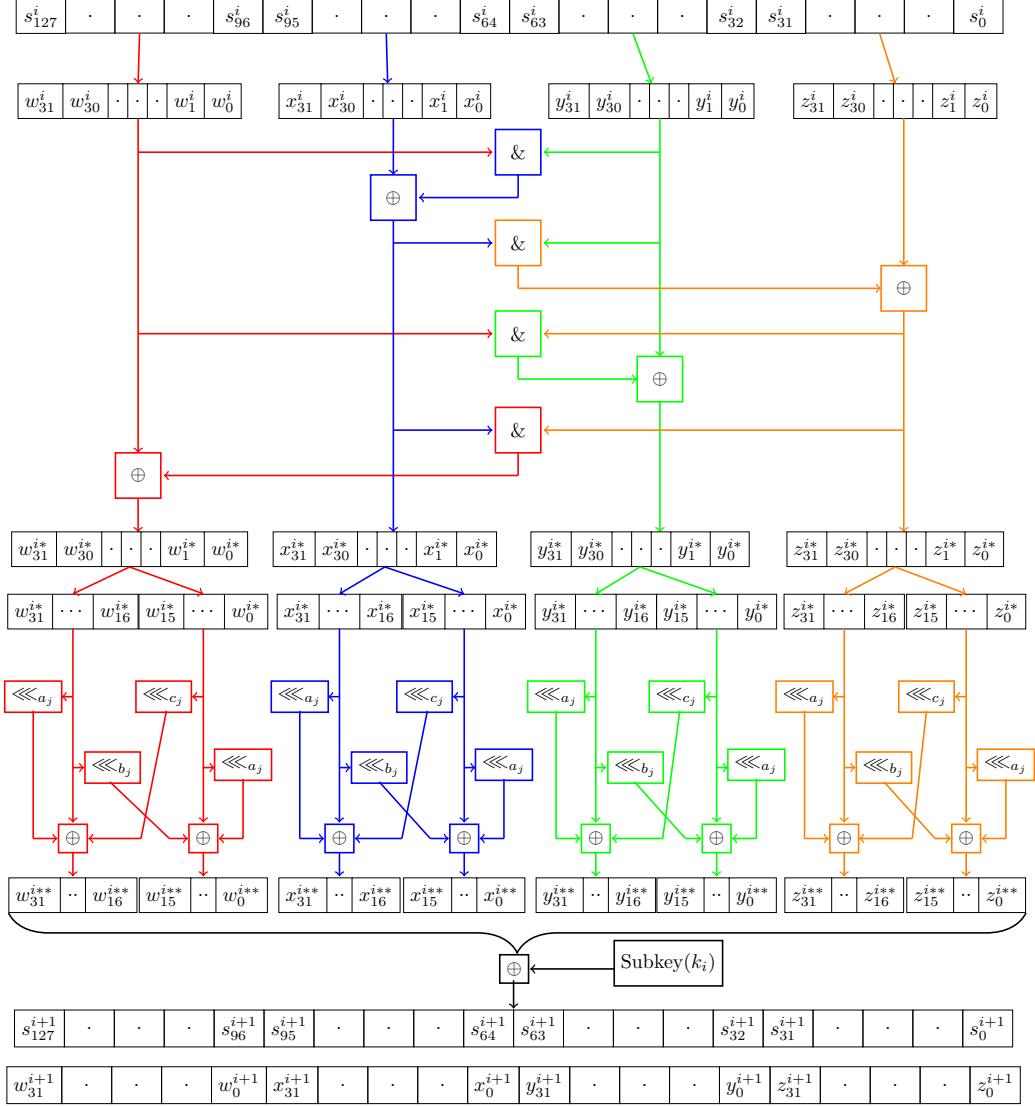


Figure 1: ARADI Round Function (R_{i+1})

2. **Word Permutation:** A permutation P is then applied to the eight words of the key state, defined as

$$P = \begin{cases} (1, 2)(5, 6), & \text{if } i \bmod 2 = 0, \\ (1, 4)(3, 6), & \text{if } i \bmod 2 = 1. \end{cases}$$

3. **Round Constant Addition:** Finally, the step index i is XORed as a round constant into the last word K_7^i .

We note that the key schedule of ARADI is an affine transformation. Consequently, all round keys are affine combinations of the master key bits.

The following diagram Figure 2 presents the key scheduling algorithm, specifically high-

lighting two consecutive rounds of the process. In this picture, each red colored box represents a specific 32-bit word of the corresponding round key. The round key for the $(r - 2)$ th round consists of the words $K_0^{r-2}, K_1^{r-2}, K_2^{r-2}, K_3^{r-2}$, while the $(r - 1)$ th round key comprises $K_4^{r-1}, K_5^{r-1}, K_6^{r-1}, K_7^{r-1}$, and the r th round key includes $K_0^r, K_1^r, K_2^r, K_3^r$. This visualization clarifies the organization of round keys within the expanded key schedule and highlights how each set of words is derived sequentially for use in their respective ARADI rounds.

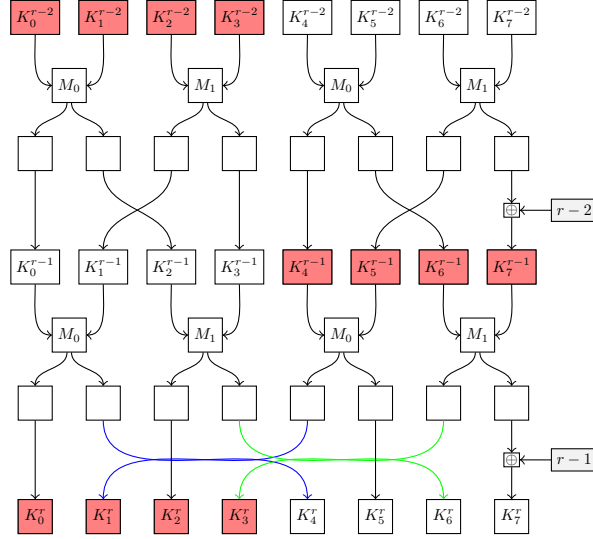


Figure 2: Diagram of Key Scheduling Algorithm

3 Differential Fault Attack on ARADI

The ARADI [12] cipher is specifically intended for unrolled implementations, where all rounds are executed in parallel. Such a design makes locating the exact injection point for a fault particularly challenging. Nonetheless, recent developments in advanced fault injection techniques [13] have shown that these difficulties can be mitigated. In practice, the timing of critical computational events often leaves observable traces in side-channel leakages. For example, power consumption profiles can be analyzed to approximate the duration of a full encryption. With this information, an attacker can synchronize fault injection to occur around the $((r - 1)/r)$ fraction of the total execution time, thereby introducing the fault in the $(r - 1)$ -th round, where r is the number of rounds in ARADI.

These improvements strengthen the assumption that adversaries can identify round boundaries even in fully unrolled implementations, and consequently inject faults such as random nibble corruptions at a desired round. This assumption is consistent with previously validated models on other lightweight ciphers. For instance, PRINCE, which also adopts an unrolled architecture, has been successfully attacked using a random nibble fault model in the 10-th round out of 12 total rounds [19, 14]. In that setting, the fault model proved sufficient for full key recovery. Therefore, it is both reasonable and timely to analyze modern unrolled block ciphers, such as ARADI, under the differential fault attack model. Following these assumptions, we consider random nibble fault injections in specific rounds of ARADI.

3.1 Attack Model and Assumptions

Our differential fault analysis is based on the following assumptions:

- **Fault Injection:** The attacker can inject random nibble faults into a word of a chosen round in ARADI. This level of fault control is realistic and commonly assumed in DFA studies.
- **Uncertain Fault Location and Value:** While a nibble fault is injected, the attacker cannot control which nibble within the word is altered, nor can they control the injected fault value. This assumption reflects realistic scenarios, where both the exact placement and outcome of a fault are inherently unpredictable.
- **Round Timing:** The attacker can identify when each round starts and ends, for example, by using side-channel information such as power traces. This makes it possible to target a specific round for fault injection.

These assumptions represent a practical attack model: the adversary does not need perfect control of the fault location or the fault value but can still exploit the induced errors to carry out an effective DFA.

3.2 Detection of Fault Injection Location

To begin the key recovery process, the first task is to select those ciphertext pairs that contain useful fault information. Assume that a random bit fault f is injected at an unknown position within a chosen word at the start of the $(r - 1)$ -th round. The effect of this fault can be tracked using the propagation patterns illustrated in Figure 3. In this figure, a bit fault f is injected at the location $z^{14}[0]$ and propagates through two consecutive round functions, namely R_{15} and R_{16} . The fault propagation path is highlighted in red in the figure. The complete set of fault propagation patterns for the last two rounds corresponding to the word z is presented in Table 3. It is important to note that the affected bits for each injection are unique and can be easily distinguished. In the second column of the table, the bits highlighted in red are of particular importance to our analysis, as the differences at these positions are always active and known to the attacker if the fault injection position is known.

It should be noted that the filtering step can be carried out without requiring any knowledge of the secret key. Since the difference patterns generated due to a fault inside a word are unique, the attacker can directly use the observed ciphertext differences to identify whether the desired single-bit fault has occurred. Specifically, when the observed difference matches one of the known propagation patterns, the corresponding ciphertext pair (original and faulty) is retained for the later stages of key recovery. On the other hand, if the difference does not correspond to any valid pattern, it is likely that more than one fault occurred simultaneously, in which case the resulting ciphertexts are discarded from consideration.

In our attack model, we consider random nibble faults rather than single-bit faults, but obviously, we need the bit-level information in the case of a nibble fault as well. Such a fault is injected at an arbitrary nibble position within a chosen word. Since ARADI does not operate on a nibble-oriented structure, we formally define what constitutes an admissible nibble difference within a word as follows:

Definition 1 (Permissible Nibble Difference (PND)). A permissible nibble difference is defined as a non-zero truncated difference with hamming weight not exceeding 4, drawn from the following set:

$$\mathcal{D}_b^\Delta = \{ *^4 0^{b-4} \gg k \mid 0 \leq k < b - 4, *^4 \in \{0, 1\}^4 \setminus \{0^4\} \},$$

where b denotes the word size.

As an illustration, when $b = 32$ (e.g., for the ARADI cipher), the set $\mathcal{D}_{32}^\Delta$ represents all possible input differences that may arise from injecting a random nibble fault into a word.

Once again, let us consider that a random Permissible Nibble Difference (PND) is injected at an arbitrary position within a chosen word at the start of the $(r - 1)$ -th round. The propagation for a particular set of these PNDs corresponding to the word z is given in Table 7. A nibble fault propagation corresponding to word z is shown in Figure 4. Again, each injected nibble fault produces a distinct difference pattern after passing through two rounds of computation, which makes it possible to uniquely identify the occurrence of the fault. Similar to the case of single-bit fault injection, the attacker can analyze the resulting ciphertext difference to determine which nibble fault has taken place.

3.3 Analysis of the Differential Fault Attack

In our key recovery attack on ARADI, we inject a random Permissible Nibble Difference (PND) at a random position chosen from the set $\mathcal{D}_{32}^\Delta$ in the word z at the beginning of $(r - 1)$ -th round. The choice of word z is deliberate. As described in Subsection 2.2, z is updated only after the words w, x , and y have been modified. This means that the effect of a fault injected into z influences the next round in a more controlled way compared to faults injected in other words.

The importance of this choice lies in how the fault propagates to the r -th round. Although the fault is injected at the beginning of the $(r - 1)$ -th round, our attack relies on generating relations at the beginning of r -th round. For this to work, we must be able to track the propagated difference in the state of the r -th round. However, due to the presence of non-linear AND operations, many of the affected bits cannot be determined with certainty; their values become unknown after propagation. Through experiments, we observed that when the fault is injected into w , or y , the state at the beginning of r -th round does not produce any useful known differences. In contrast, when the fault is injected into x or z , some bits of this state remain predictable. This property motivates our choice of the word z as the target for the attack; a similar strategy could also be applied to the word x . To reproduce the full attack targeting the word x , consult Table 8 and Table 9. For the remainder of the paper, we present and analyze the attack assuming fault injections into the word z .

Our objective is to obtain known differences at the beginning of the r -th round by injecting a fault at the beginning of the $(r - 1)$ -th round. When the fault is introduced into the word z at the start of round $(r - 1)$, the difference propagates through one round and spreads across the words w, y , and z . However, only the differences in z remain known, while those appearing in w and y are unknown due to the effect of the non-linear operations. As a result, the differences in w and y cannot be directly used to construct useful equations.

It is worth noting that inactive differences, which appear in all four words because of the fault injection, could in principle also be used. In practice, however, when attempting to compute the corresponding XOR-difference equations, these cancel out because the same variables appear in both the original and the faulty equations. Consequently, we are restricted to forming equations only from the active differences in z^{r-1} .

In our notation, $a^i[j]$ represents the j -th bit of the word a in round $(i + 1)$. Similarly, $a^{i*}[j]$ denotes the j -th bit of the word a obtained after applying the nonlinear layer π to the state s^i , as illustrated in Figure 1. Now, to link a bit of z^{r-1} with the last-round sub key bits and the ciphertext, we proceed by establishing the following relation:

$$w^i[j] = w^{i*}[j] \oplus (x^{i*}[j] \& z^{i*}[j]) \quad (1)$$

$$\begin{aligned}
y^i[j] &= y^{i*}[j] \oplus (w^i[j] \& z^{i*}[j]) \\
&= y^{i*}[j] \oplus ((w^{i*}[j] \oplus (x^{i*}[j] \& z^{i*}[j])) \& z^{i*}[j]) \\
&\quad \text{(using Equation 1)} \\
&= y^{i*}[j] \oplus (w^{i*}[j] \& z^{i*}[j]) \oplus (x^{i*}[j] \& z^{i*}[j])
\end{aligned} \tag{2}$$

$$\begin{aligned}
z^i[j] &= z^{i*}[j] \oplus (x^{i*}[j] \& y^i[j]) \\
&= z^{i*}[j] \oplus (x^{i*}[j] \& y^{i*}[j]) \oplus (x^{i*}[j] \& w^{i*}[j] \& z^{i*}[j]) \\
&\quad \oplus (x^{i*}[j] \& z^{i*}[j]) \text{(using Equation 2)}
\end{aligned} \tag{3}$$

Now, express each round key as:

$$\tau_{k_i} = (k_w^i[31], \dots, k_w^i[0] \parallel k_x^i[31], \dots, k_x^i[0] \parallel k_y^i[31], \dots, k_y^i[0] \parallel k_z^i[31], \dots, k_z^i[0])$$

Therefore, according to the round rotation constants $\{a_j, b_j, c_j\}$

$$z^i[j] = z^{i*}[j] \oplus (x^{i*}[j] \& y^{i*}[j]) \oplus (x^{i*}[j] \& w^{i*}[j] \& z^{i*}[j]) \oplus (x^{i*}[j] \& z^{i*}[j]),$$

Where,

$$\begin{cases}
z^{i*}[j] = (z^{i+1}[j] \oplus z^{i+1}[j + a_i] \oplus z^{i+1}[j + 16 + c_i]) \\
\quad \oplus (k_z^{i+1}[j] \oplus k_z^{i+1}[j + a_i] \oplus k_z^{i+1}[j + 16 + c_i]), \\
y^{i*}[j] = (y^{i+1}[j] \oplus y^{i+1}[j + a_i] \oplus y^{i+1}[j + 16 + c_i]) \\
\quad \oplus (k_y^{i+1}[j] \oplus k_y^{i+1}[j + a_i] \oplus k_y^{i+1}[j + 16 + c_i]), \\
x^{i*}[j] = (x^{i+1}[j] \oplus x^{i+1}[j + a_i] \oplus x^{i+1}[j + 16 + c_i]) \\
\quad \oplus (k_x^{i+1}[j] \oplus k_x^{i+1}[j + a_i] \oplus k_x^{i+1}[j + 16 + c_i]), \\
w^{i*}[j] = (w^{i+1}[j] \oplus w^{i+1}[j + a_i] \oplus w^{i+1}[j + 16 + c_i]) \\
\quad \oplus (k_w^{i+1}[j] \oplus k_w^{i+1}[j + a_i] \oplus k_z^{i+1}[j + 16 + c_i]) \\
\quad \forall j \in \{0, \dots, 15\}
\end{cases}$$

And,

$$\begin{cases}
z^{i*}[j] = (z^{i+1}[j] \oplus z^{i+1}[j + a_i] \oplus z^{i+1}[j - 16 + b_i]) \\
\quad \oplus (k_z^{i+1}[j] \oplus k_z^{i+1}[j + a_i] \oplus k_z^{i+1}[j - 16 + b_i]), \\
y^{i*}[j] = (y^{i+1}[j] \oplus y^{i+1}[j + a_i] \oplus y^{i+1}[j - 16 + b_i]) \\
\quad \oplus (k_y^{i+1}[j] \oplus k_y^{i+1}[j + a_i] \oplus k_y^{i+1}[j - 16 + b_i]), \\
x^{i*}[j] = (x^{i+1}[j] \oplus x^{i+1}[j + a_i] \oplus x^{i+1}[j - 16 + b_i]) \\
\quad \oplus (k_x^{i+1}[j] \oplus k_x^{i+1}[j + a_i] \oplus k_x^{i+1}[j - 16 + b_i]), \\
w^{i*}[j] = (w^{i+1}[j] \oplus w^{i+1}[j + a_i] \oplus w^{i+1}[j - 16 + b_i]) \\
\quad \oplus (k_w^{i+1}[j] \oplus k_w^{i+1}[j + a_i] \oplus k_z^{i+1}[j - 16 + b_i]) \\
\quad \forall j \in \{16, \dots, 31\}
\end{cases}$$

For example, if we want a difference equation at the beginning of the round r in the LSB of the word z^{15} , we have

$$z^{15}[0] = z^{15*}[0] \oplus (x^{15*}[0] \& y^{15*}[0]) \oplus (x^{15*}[0] \& w^{15*}[0] \& z^{15*}[0]) \oplus (x^{15*}[0] \& z^{15*}[0])$$

for the original ciphertext

$$(w^{16}[31], \dots, w^{16}[1], w^{16}[0], x^{16}[31], \dots, x^{16}[1], x^{16}[0], y_3^{16}[1], \dots, y^{16}[1], y^{16}[0], z^{16}[31], \dots, z^{16}[1], z^{16}[0])$$

and key

$$(k_w^{16}[31], \dots, k_w^{16}[0] \parallel k_x^{16}[31], \dots, k_x^{16}[0] \parallel k_y^{16}[31], \dots, k_y^{16}[0] \parallel k_z^{16}[31], \dots, k_z^{16}[0])$$

where,

$$\begin{cases} z^{15*}[0] = (z^{16}[0] \oplus z^{16}[8] \oplus z^{16}[23]) \oplus (k_z^{16}[0] \oplus k_z^{16}[8] \oplus k_z^{16}[23]), \\ y^{15*}[0] = (y^{16}[0] \oplus y^{16}[8] \oplus y^{16}[23]) \oplus (k_y^{16}[0] \oplus k_y^{16}[8] \oplus k_y^{16}[23]), \\ x^{15*}[0] = (x^{16}[0] \oplus x^{16}[8] \oplus x^{16}[23]) \oplus (k_x^{16}[0] \oplus k_x^{16}[8] \oplus k_x^{16}[23]), \\ w^{15*}[0] = (w^{16}[0] \oplus w^{16}[8] \oplus w^{16}[23]) \oplus (k_w^{16}[0] \oplus k_w^{16}[8] \oplus k_w^{16}[23]) \end{cases}$$

And similarly for the faulty ciphertext

$$(w'^{16}[31], \dots, w'^{16}[1], w'^{16}[0], x'^{16}[31], \dots, x'^{16}[1], x'^{16}[0], y_3'^{16}[1], \dots, y'^{16}[1], y'^{16}[0], z'^{16}[31], \dots, z'^{16}[1], z'^{16}[0])$$

we have $z'^{15}[0]$ and therefore, we generate $\Delta z^{15}[0] = z^{15}[0] \oplus z'^{15}[0]$ which involves the sub key bits:

$$\{k_w^{16}[0], k_w^{16}[8], k_w^{16}[23], k_x^{16}[0], k_x^{16}[8], k_x^{16}[23], k_y^{16}[0], k_y^{16}[8], k_y^{16}[23], k_z^{16}[0], k_z^{16}[8], k_z^{16}[23]\}$$

The complete list for the involve sub key bits corresponding to the bit positions of z^{15} has been given below:

- $\Delta z^{15}[0] : \{k_{w,x,y,z}^{16}[0], k_{w,x,y,z}^{16}[8], k_{w,x,y,z}^{16}[23]\}$
- $\Delta z^{15}[1] : \{k_{w,x,y,z}^{16}[1], k_{w,x,y,z}^{16}[9], k_{w,x,y,z}^{16}[24]\}$
- $\Delta z^{15}[2] : \{k_{w,x,y,z}^{16}[2], k_{w,x,y,z}^{16}[10], k_{w,x,y,z}^{16}[25]\}$
- $\Delta z^{15}[3] : \{k_{w,x,y,z}^{16}[3], k_{w,x,y,z}^{16}[11], k_{w,x,y,z}^{16}[26]\}$
- $\Delta z^{15}[4] : \{k_{w,x,y,z}^{16}[4], k_{w,x,y,z}^{16}[12], k_{w,x,y,z}^{16}[27]\}$
- $\Delta z^{15}[5] : \{k_{w,x,y,z}^{16}[5], k_{w,x,y,z}^{16}[13], k_{w,x,y,z}^{16}[28]\}$
- $\Delta z^{15}[6] : \{k_{w,x,y,z}^{16}[6], k_{w,x,y,z}^{16}[14], k_{w,x,y,z}^{16}[29]\}$
- $\Delta z^{15}[7] : \{k_{w,x,y,z}^{16}[7], k_{w,x,y,z}^{16}[15], k_{w,x,y,z}^{16}[30]\}$
- $\Delta z^{15}[8] : \{k_{w,x,y,z}^{16}[0], k_{w,x,y,z}^{16}[8], k_{w,x,y,z}^{16}[31]\}$
- $\Delta z^{15}[9] : \{k_{w,x,y,z}^{16}[1], k_{w,x,y,z}^{16}[9], k_{w,x,y,z}^{16}[16]\}$
- $\Delta z^{15}[10] : \{k_{w,x,y,z}^{16}[2], k_{w,x,y,z}^{16}[10], k_{w,x,y,z}^{16}[25]\}$
- $\Delta z^{15}[11] : \{k_{w,x,y,z}^{16}[3], k_{w,x,y,z}^{16}[11], k_{w,x,y,z}^{16}[18]\}$
- $\Delta z^{15}[12] : \{k_{w,x,y,z}^{16}[4], k_{w,x,y,z}^{16}[12], k_{w,x,y,z}^{16}[19]\}$
- $\Delta z^{15}[13] : \{k_{w,x,y,z}^{16}[5], k_{w,x,y,z}^{16}[13], k_{w,x,y,z}^{16}[20]\}$
- $\Delta z^{15}[14] : \{k_{w,x,y,z}^{16}[6], k_{w,x,y,z}^{16}[14], k_{w,x,y,z}^{16}[21]\}$
- $\Delta z^{15}[15] : \{k_{w,x,y,z}^{16}[7], k_{w,x,y,z}^{16}[15], k_{w,x,y,z}^{16}[22]\}$
- $\Delta z^{15}[16] : \{k_{w,x,y,z}^{16}[9], k_{w,x,y,z}^{16}[16], k_{w,x,y,z}^{16}[24]\}$
- $\Delta z^{15}[17] : \{k_{w,x,y,z}^{16}[10], k_{w,x,y,z}^{16}[17], k_{w,x,y,z}^{16}[25]\}$
- $\Delta z^{15}[18] : \{k_{w,x,y,z}^{16}[11], k_{w,x,y,z}^{16}[18], k_{w,x,y,z}^{16}[26]\}$
- $\Delta z^{15}[19] : \{k_{w,x,y,z}^{16}[12], k_{w,x,y,z}^{16}[19], k_{w,x,y,z}^{16}[27]\}$

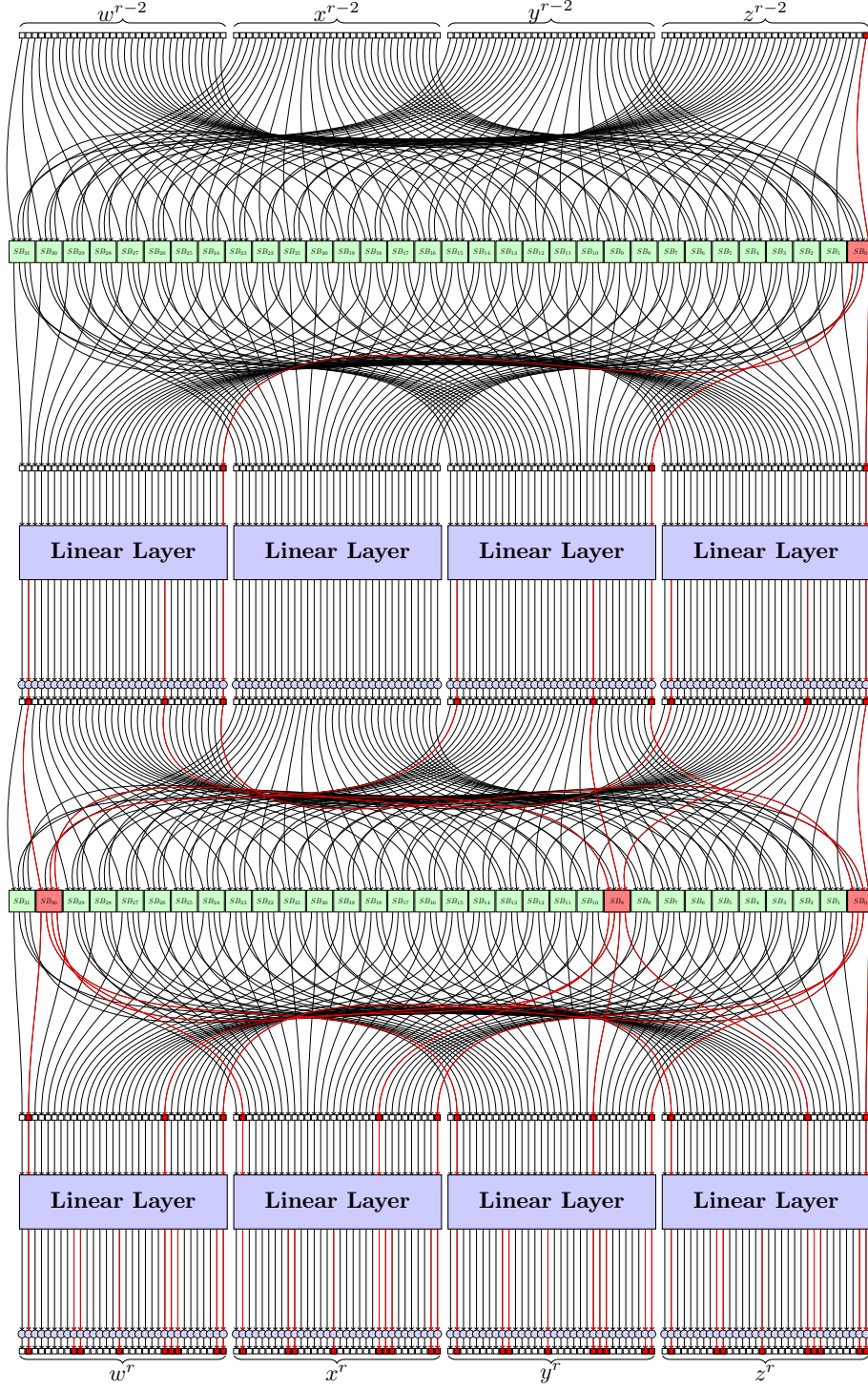


Figure 3: Diagram of Bit Fault Propagation through the last two rounds of ARADI.

- $\Delta z^{15}[20] : \{k_{w,x,y,z}^{16}[13], k_{w,x,y,z}^{16}[20], k_{w,x,y,z}^{16}[28]\}$
- $\Delta z^{15}[21] : \{k_{w,x,y,z}^{16}[14], k_{w,x,y,z}^{16}[21], k_{w,x,y,z}^{16}[29]\}$

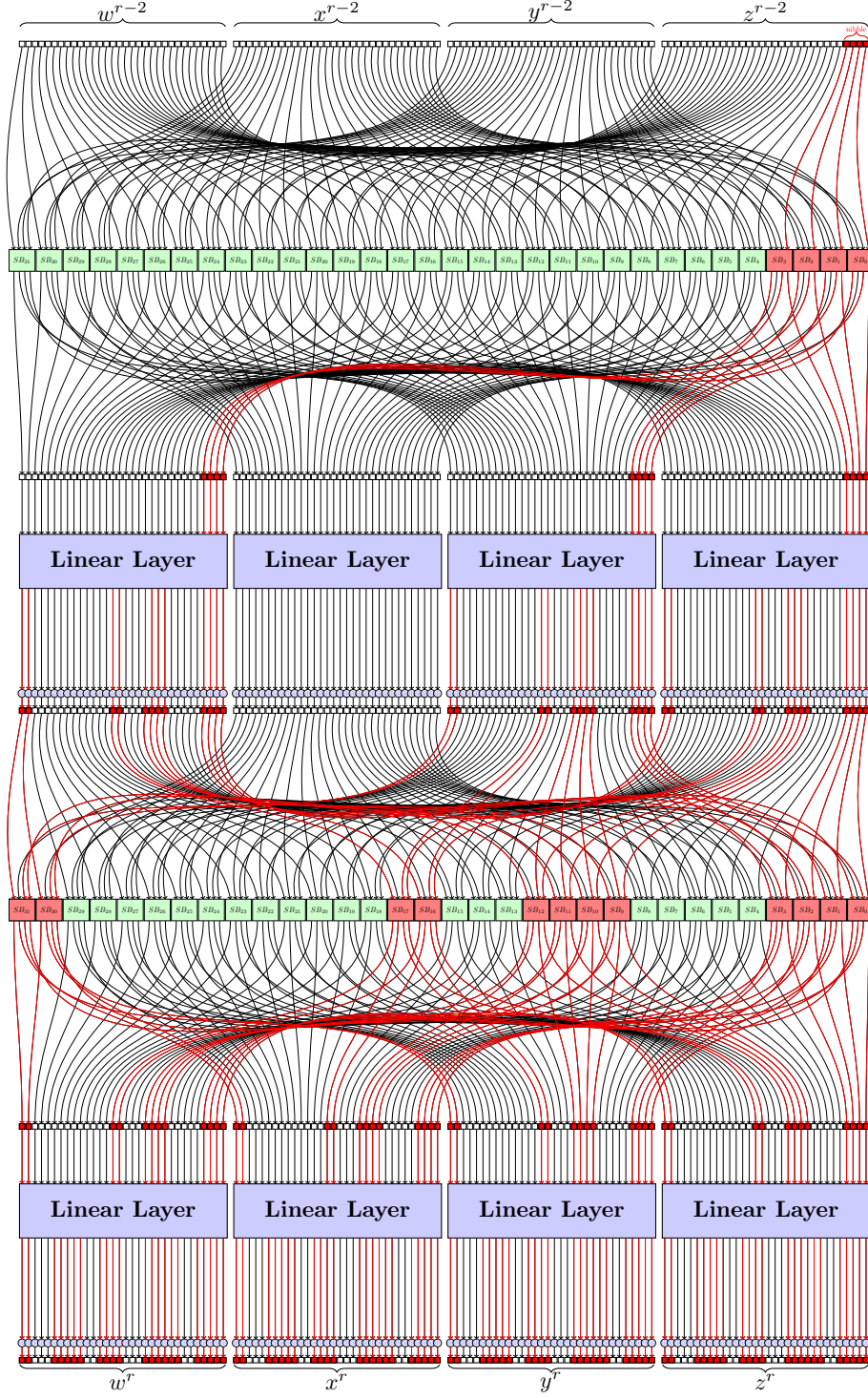


Figure 4: Diagram of Nibble Fault Propagation through the last two rounds of ARADI

- $\Delta z^{15}[22] : \{k_{w,x,y,z}^{16}[15], k_{w,x,y,z}^{16}[22], k_{w,x,y,z}^{16}[30]\}$
- $\Delta z^{15}[23] : \{k_{w,x,y,z}^{16}[0], k_{w,x,y,z}^{16}[23], k_{w,x,y,z}^{16}[31]\}$

- $\Delta z^{15}[24] : \{k_{w,x,y,z}^{16}[1], k_{w,x,y,z}^{16}[16], k_{w,x,y,z}^{16}[24]\}$
- $\Delta z^{15}[25] : \{k_{w,x,y,z}^{16}[2], k_{w,x,y,z}^{16}[17], k_{w,x,y,z}^{16}[25]\}$
- $\Delta z^{15}[26] : \{k_{w,x,y,z}^{16}[3], k_{w,x,y,z}^{16}[18], k_{w,x,y,z}^{16}[26]\}$
- $\Delta z^{15}[27] : \{k_{w,x,y,z}^{16}[4], k_{w,x,y,z}^{16}[19], k_{w,x,y,z}^{16}[27]\}$
- $\Delta z^{15}[28] : \{k_{w,x,y,z}^{16}[5], k_{w,x,y,z}^{16}[20], k_{w,x,y,z}^{16}[28]\}$
- $\Delta z^{15}[29] : \{k_{w,x,y,z}^{16}[6], k_{w,x,y,z}^{16}[21], k_{w,x,y,z}^{16}[29]\}$
- $\Delta z^{15}[30] : \{k_{w,x,y,z}^{16}[7], k_{w,x,y,z}^{16}[22], k_{w,x,y,z}^{16}[30]\}$
- $\Delta z^{15}[31] : \{k_{w,x,y,z}^{16}[8], k_{w,x,y,z}^{16}[23], k_{w,x,y,z}^{16}[31]\}$

Before attempting full master-key recovery, we first target the subkey of the last round

$$(k_w^{16}[31], \dots, k_w^{16}[0] \parallel k_x^{16}[31], \dots, k_x^{16}[0] \parallel k_y^{16}[31], \dots, k_y^{16}[0] \parallel k_z^{16}[31], \dots, k_z^{16}[0])$$

We partition the last-round subkey into eight disjoint groups, each containing 16 key bits. The groups are listed as follows:

1. $\{k_{w,x,y,z}^{16}[0], k_{w,x,y,z}^{16}[8], k_{w,x,y,z}^{16}[23], k_{w,x,y,z}^{16}[31]\}$
2. $\{k_{w,x,y,z}^{16}[1], k_{w,x,y,z}^{16}[9], k_{w,x,y,z}^{16}[16], k_{w,x,y,z}^{16}[24]\}$
3. $\{k_{w,x,y,z}^{16}[2], k_{w,x,y,z}^{16}[10], k_{w,x,y,z}^{16}[17], k_{w,x,y,z}^{16}[25]\}$
4. $\{k_{w,x,y,z}^{16}[3], k_{w,x,y,z}^{16}[11], k_{w,x,y,z}^{16}[18], k_{w,x,y,z}^{16}[26]\}$
5. $\{k_{w,x,y,z}^{16}[4], k_{w,x,y,z}^{16}[12], k_{w,x,y,z}^{16}[19], k_{w,x,y,z}^{16}[27]\}$
6. $\{k_{w,x,y,z}^{16}[5], k_{w,x,y,z}^{16}[13], k_{w,x,y,z}^{16}[20], k_{w,x,y,z}^{16}[28]\}$
7. $\{k_{w,x,y,z}^{16}[6], k_{w,x,y,z}^{16}[14], k_{w,x,y,z}^{16}[21], k_{w,x,y,z}^{16}[29]\}$
8. $\{k_{w,x,y,z}^{16}[7], k_{w,x,y,z}^{16}[15], k_{w,x,y,z}^{16}[22], k_{w,x,y,z}^{16}[30]\}$

Here, each $k_{w,x,y,z}^r[j]$ collectively denotes the set $\{k_w^r[j], k_x^r[j], k_y^r[j], k_z^r[j]\}$ for brevity. Note that since the groups of subkey bits are distinct, each group can be recovered independently. We begin by discussing the recovery of the first group, which consists of the following 16 subkey bits:

$$\{k_w^{16}[0], k_w^{16}[8], k_w^{16}[23], k_w^{16}[31], k_x^{16}[0], k_x^{16}[8], k_x^{16}[23], k_x^{16}[31], \\ k_y^{16}[0], k_y^{16}[8], k_y^{16}[23], k_y^{16}[31], k_z^{16}[0], k_z^{16}[8], k_z^{16}[23], k_z^{16}[31]\}.$$

To recover these 16 bits, we must construct a sufficient number of difference equations. If the equations were linear, then in theory, 16 equations would suffice. However, since the equations are nonlinear, more than 16 equations are required. Now, to obtain enough equations, we rely on introducing known differences at specific positions of z^{r-1} , ensuring that the resulting equations involve the above 16 key bits.

Note that if we generate differences at $\Delta z^{15}[0]$, $\Delta z^{15}[8]$, $\Delta z^{15}[23]$, and $\Delta z^{15}[31]$, and construct a sufficient number of equations, we can uniquely determine the 16 subkey bits of this group. These difference equations involve only those 16 key bits.

For each 16-bit subkey group, we construct the corresponding set of differences shown below; these sets are used later in the actual attack algorithm.

- **Group 1:** $\{\Delta z^{15}[0], \Delta z^{15}[8], \Delta z^{15}[23], \Delta z^{15}[31]\}$
- **Group 2:** $\{\Delta z^{15}[1], \Delta z^{15}[9], \Delta z^{15}[16], \Delta z^{15}[24]\}$
- **Group 3:** $\{\Delta z^{15}[2], \Delta z^{15}[10], \Delta z^{15}[17], \Delta z^{15}[25]\}$

- **Group 4:** $\{\Delta z^{15}[3], \Delta z^{15}[11], \Delta z^{15}[18], \Delta z^{15}[26]\}$
- **Group 5:** $\{\Delta z^{15}[4], \Delta z^{15}[12], \Delta z^{15}[19], \Delta z^{15}[27]\}$
- **Group 6:** $\{\Delta z^{15}[5], \Delta z^{15}[13], \Delta z^{15}[20], \Delta z^{15}[28]\}$
- **Group 7:** $\{\Delta z^{15}[6], \Delta z^{15}[14], \Delta z^{15}[21], \Delta z^{15}[29]\}$
- **Group 8:** $\{\Delta z^{15}[7], \Delta z^{15}[15], \Delta z^{15}[22], \Delta z^{15}[30]\}$

3.4 Attack Procedure

The high-level procedure of our key-recovery attack is summarized in [algorithm 1](#). In short, for each injected Permissible Nibble Difference (PND) we collect the original and faulty ciphertext pair, use the known fault-propagation patterns to determine which bits of z^{r-2} were corrupted, map those corrupted positions to the positions in z^{r-1} that yield *known* active differences, and form XOR-difference equations relating those known differences to the unknown key bits. Equations produced from multiple PNDs are grouped so that each group contains equations involving a specific 16-bit subkey. We solve each group independently (using a SAT solver), gather candidate subkey solutions, and finally perform an exhaustive search over the remaining possible keys to obtain the unique full key.

Key points:

- We inject f random PNDs into random nibble positions of z^{r-2} and collect the corresponding original and faulty ciphertext pairs.
- For each ciphertext pair, the observed difference pattern at (w^r, x^r, y^r, z^r) is examined to infer which bits of z^{r-2} were affected (using the bit-pattern [Table 3](#)).
- From the affected bits of z^{r-2} we determine which positions in z^{r-1} have known active differences (using the bit-pattern [Table 3](#), where the active known difference are high lighted with red colour) and build the corresponding difference equations (each equation involves ciphertext bits and a subset of key bits).
- Collected equations are partitioned into 8 groups; each group relates to a distinct 16-bit subkey. Each group's equation set is solved (e.g., using a SAT solver) to obtain candidate assignments for those 16 bits.
- Finally, the remaining bits are resolved by an exhaustive search over the small remaining possible keys to yield the unique 128-bit key.

ILLUSTRATIVE EXAMPLE OF THE ATTACK PROCEDURE: To demonstrate the attack procedure, consider the case where the adversary injects ten random Permissible Nibble Differences (PNDs) into z^{14} . For each injection we collect the original and faulty ciphertext pair, inspect the observed difference pattern at $(w^{16}, x^{16}, y^{16}, z^{16})$, and consult [Table 3](#) to identify which bit positions of z^{15} become *known* (active) as a result. The ten injected PNDs (order not important here) are

$$\{0x0000000f, 0x00000700, 0x00300000, 0x00e00000, 0x00001000, \\ 0x00900000, 0x00a00000, 0x00078000, 0x0c000000, 0x3c000000\}$$

Each PND affects up to four bit positions of z^{14} , which in turn map to specific red (active)

Algorithm 1: Recovering the 128-bit Key

Input : Required number of nibble faults f
Output : Recovered 128-bit key

- 1 Initialize an empty list (or multimap) $\mathcal{E}$ of equations indexed by bit positions $0, \dots, 31$;
- 2 **for** $i \leftarrow 1$ **to** f **do**
- 3 Inject a random PND f_i at a uniformly random nibble position in z^{r-2} ;
- 4 Collect the original and faulty ciphertext pair (C, C') ;
- 5 Determine the affected bit positions of z^{r-2} using the pattern at (w^r, x^r, y^r, z^r) (see bit-pattern Table 3);
- 6 From the affected z^{r-2} bits, infer which positions j in z^{r-1} yield known active differences (using the bit-pattern Table 3);
- 7 **for** each such position j **do**
- 8 Construct the XOR-difference equation $E_{i,j}$ that relates known ciphertext bits and the unknown key variables;
- 9 Append $E_{i,j}$ to $\mathcal{E}[j]$;
- 10 **end**
- 11 **end**
- 12 Partition the 32 positions into 8 groups (each group covering the 4 positions relevant to a 16-bit subkey);
- 13 **for** each group $g = 1, \dots, 8$ **do**
- 14 Collect all equations from $\mathcal{E}$ that belong to group g ;
- 15 Encode the collected equations into the chosen solver format (e.g., SAT);
- 16 Run the solver to obtain candidate assignments for the 16 subkey bits of group g ;
- 17 Store the candidate solutions for group g ;
- 18 **end**
- 19 Combine candidate group-wise solutions and perform exhaustive search over any remaining undecided bits to find the unique 128-bit key;
- 20 **return** *Recovered 128-bit key*

entries in the z^{15} row of Table 3. The PND-to- z^{14} bit mapping is

$$\left\{ \begin{array}{l} \{0x0000000f \rightarrow \{3, 2, 1, 0\}\}, \\ \{0x00000700 \rightarrow \{10, 9, 8\}\}, \\ \{0x00300000 \rightarrow \{21, 20\}\}, \\ \{0x00e00000 \rightarrow \{23, 22, 21\}\}, \\ \{0x00001000 \rightarrow \{12\}\}, \\ \{0x00900000 \rightarrow \{23, 20\}\}, \\ \{0x00a00000 \rightarrow \{23, 21\}\}, \\ \{0x00078000 \rightarrow \{18, 17, 16, 15\}\}, \\ \{0x0c000000 \rightarrow \{27, 26\}\}, \\ \{0x3c000000 \rightarrow \{29, 28, 27, 26\}\} \end{array} \right.$$

Using the propagation rules in Table 3, each of the above PNDs yields a set of active

indices in z^{15} . The full active-bit mappings for these ten PNDs are respectively

$$\left\{ \begin{array}{l} \{z^{15}[0, 9, 30], z^{15}[1, 10, 31], z^{15}[2, 11, 16], z^{15}[3, 12, 17]\}, \\ \{z^{15}[8, 22, 1], z^{15}[9, 23, 2], z^{15}[10, 24, 3]\}, \\ \{z^{15}[20, 29, 8], z^{15}[21, 30, 9]\}, \\ \{z^{15}[21, 30, 9], z^{15}[22, 31, 10], z^{15}[23, 16, 11]\}, \\ \{z^{15}[12, 26, 5]\}, \\ \{z^{15}[20, 29, 8], z^{15}[23, 16, 11]\}, \\ \{z^{15}[21, 30, 9], z^{15}[23, 16, 11]\}, \\ \{z^{15}[8, 15, 29], z^{15}[16, 25, 4], z^{15}[17, 26, 5], z^{15}[18, 27, 6]\}, \\ \{z^{15}[14, 19, 26], z^{15}[15, 20, 27]\}, \\ \{z^{15}[14, 19, 26], z^{15}[15, 20, 27], z^{15}[28, 21, 0], z^{15}[29, 22, 1]\} \end{array} \right.$$

Table 4: Frequency of $z^{15}[i]$ in the list

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	2	3	2	2	1	2	1	0	4	5	3	5	2	0	2	3
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
count	5	2	1	2	4	4	3	4	1	1	4	3	1	4	4	2

Table 5: Total number of equations corresponding to each group

Group	1	2	3	4	5	6	7	8
Total Equations	12	14	8	12	8	7	11	10

Each listed z^{15} index above denotes a *known* bit position produced by the corresponding PND; these known positions are then translated into XOR-difference equations that relate ciphertext bits to unknown key variables.

Table 4 summarizes how many times each $z^{15}[i]$ became known across the ten injections. These frequencies determine how many independent difference equations are available at each bit position and are used to assemble group-wise equation sets. The total number of equations obtained for each group (from the ten PNDs) is shown in Table 5.

Finally, for each group, we submit the corresponding set of difference equations to a SAT solver to obtain candidate assignments for the 16-bit subkey associated with that group; the candidate solutions from all groups are combined, and any remaining key bits are resolved by exhaustive search to recover the unique 128-bit key.

3.5 Master Key Recovery

The objective of our analysis is to recover the master key. Since each round key consists of 128 bits, recovering a single round key is insufficient to find the complete 256-bit master key.

Based on the key scheduling algorithm described above and illustrated in Figure 2, the recovery process of the master key from round keys can be expressed as follows:

1. Recovery from the r -th round key: Obtaining the r th round key words $\{K_0^r, K_1^r, K_2^r, K_3^r\}$ alone is insufficient to reconstruct the master key:

$$\text{i.e., } \{K_0^r, K_1^r, K_2^r, K_3^r\} \not\Rightarrow \text{Master Key.}$$

2. Recovery from the $(r - 1)$ -th round key: If the $(r - 1)$ -th round key words $K_4^{r-1}, K_5^{r-1}, K_6^{r-1}, K_7^{r-1}$ are available, then

$$\{K_4^{r-1}, K_5^{r-1}, K_6^{r-1}, K_7^{r-1}\} \Rightarrow \{K_1^r, K_3^r, K_5^r, K_7^r\}.$$

So, we recover $\{K_5^r, K_7^r\}$ and additionally, $\{K_1^r, K_3^r\}$ can be verified.

3. Recovery from the $(r - 2)$ -th round key: From the $(r - 2)$ -th round key $\{K_0^{r-2}, K_1^{r-2}, K_2^{r-2}, K_3^{r-2}\}$, we can derive $\{K_0^{r-1}, K_1^{r-1}, K_2^{r-1}, K_3^{r-1}\}$, which further yields $\{K_0^r, K_2^r, K_4^r, K_6^r\}$.

$$\text{i.e., } \{K_0^{r-2}, K_1^{r-2}, K_2^{r-2}, K_3^{r-2}\} \Rightarrow \{K_0^r, K_2^r, K_4^r, K_6^r\}.$$

Consequently, we recover $\{K_4^r, K_6^r\}$ and validate $\{K_0^r, K_2^r\}$.

4. Complete r -th round key schedule state reconstruction: Combining the results from steps (2) and (3), the entire r -th round key state can be determined:

$$K^r = \{K_0^r, K_1^r, K_2^r, K_3^r, K_4^r, K_5^r, K_6^r, K_7^r\}.$$

5. Master key recovery: Once all the key words of the r -th round key state (K^r) are known, the inverse of the key scheduling algorithm can be applied to reconstruct the master key (K^0).

$$\begin{aligned} \text{i.e., } K^r &= \{K_0^r, K_1^r, K_2^r, K_3^r, K_4^r, K_5^r, K_6^r, K_7^r\} \Rightarrow \\ &\{K_0^0, K_1^0, K_2^0, K_3^0, K_4^0, K_5^0, K_6^0, K_7^0\} = K^0 \end{aligned}$$

Therefore, it follows that the recovery of three consecutive round keys is sufficient to reconstruct the entire master key. Once the last-round subkey has been recovered, the state s^{r-1} can be derived from s^r by undoing the last round using the recovered subkey. The same fault-analysis procedure is then applied to recover the subkey of round $r - 1$. Repeating this step one more time yields the subkey of round $r - 2$; combining these recovered round subkeys gives the full master key.

3.6 Complexity Analysis & Simulation Results

In this section, we determine the number of random permissible nibble differences (PNDs) required to recover the master key, and we present the time complexity together with simulation results.

Our key recovery proceeds iteratively over the last three rounds. First, we recover the round-16 subkey (the last round key). Once the last round key is obtained, it can be used to partially decrypt both the original and faulty ciphertexts by one round, thus exposing the state at the end of round 15. This allows us to apply the same procedure used for the last round: inject PNDs at the start of round 14, collect ciphertext pairs, form difference equations for round 15, and solve for the round-15 subkey. Repeating this process one more time (injecting at the start of round 13 and generating equations for round 14) yields the round-14 subkey.

Because each round is treated with essentially the same equation-generation and solver pipeline, the average effort required to recover each round subkey is comparable. Consequently, the total effort to recover the three-round subkeys is roughly three times the cost of recovering a single-round subkey (up to minor constant factors due to bookkeeping and solver overhead). Here, we quantify the number of PNDs needed per round and give empirical timings from our simulation study.

Table 6: Probabilities to generate an equation at $z^{15}[i]$.

$z^{15}[0]$	0.165	$z^{15}[1]$	0.166	$z^{15}[2]$	0.165	$z^{15}[3]$	0.166
$z^{15}[4]$	0.221	$z^{15}[5]$	0.221	$z^{15}[6]$	0.221	$z^{15}[7]$	0.221
$z^{15}[8]$	0.221	$z^{15}[9]$	0.165	$z^{15}[10]$	0.184	$z^{15}[11]$	0.202
$z^{15}[12]$	0.221	$z^{15}[13]$	0.221	$z^{15}[14]$	0.221	$z^{15}[15]$	0.221
$z^{15}[16]$	0.202	$z^{15}[17]$	0.221	$z^{15}[18]$	0.221	$z^{15}[19]$	0.221
$z^{15}[20]$	0.221	$z^{15}[21]$	0.221	$z^{15}[22]$	0.202	$z^{15}[23]$	0.184
$z^{15}[24]$	0.166	$z^{15}[25]$	0.221	$z^{15}[26]$	0.221	$z^{15}[27]$	0.221
$z^{15}[28]$	0.221	$z^{15}[29]$	0.202	$z^{15}[30]$	0.129	$z^{15}[31]$	0.129

First, we determine how many equations are required to recover the key bits associated with each group. Since each group is distinct, the subkey bits can be recovered one group at a time.

To recover the subkey bits of the first group, we require at least 16 equations involving the positions $\{z^{15}[0], z^{15}[8], z^{15}[23], z^{15}[31]\}$. We inject random PIDs into z^{14} , and collect equations of the form $\{\Delta z^{15}[0], \Delta z^{15}[8], \Delta z^{15}[23], \Delta z^{15}[31]\}$. These equations, which together involve 16 key bits, are then provided to the SAT solver. We employed CryptoMiniSat [20] as the SAT solver for conducting our simulations.

The solver results show the following:

- With 27 equations, the keyspace reduces from 2^{16} to 2^4 .
- With 36 equations, the keyspace reduces further to 2^2 .
- With 45 equations, we obtain a unique solution for the 16-bit key.

This procedure is repeated across all eight groups, and we observed consistent results on average. To balance the tradeoff between time complexity and number of faults, we tolerate a slightly larger time complexity in order to minimize fault injections. Thus, we choose 27 equations per group, leading to a time complexity of $(2^4)^8 = 2^{32}$, which is practical.

Next, we calculate the number of fault injections required. From the probabilities of generating known active differences in the bits of z^{15} (see Table 6), it can be observed that, on average, the probability of obtaining an equation in any bit of z^{15} is approximately the same. Therefore, to recover the key bits with practical time complexity and minimal faults, we require about $\frac{27}{4} \approx 7$ equations per bit of z^{15} . According to our simulation results, to achieve this, a maximum of 36 random PID injections into z^{14} are sufficient. Therefore, recovering the last-round subkey requires 36 random PND injections and a time complexity of 2^{32} . Consequently, recovering the full master key requires $36 \times 3 = 108$ random PND injections in total, with an overall time complexity of 3×2^{32} .

3.7 Countermeasures

The analysis presented in this paper highlights the vulnerability of the ARADI cipher due to differential fault attacks based on permissible nibble differences (PNDs), where the introduction of specific PNDs reduces the effective key search space and significantly improves the success probability of the attack. To mitigate such vulnerabilities, several possible countermeasures can be considered.

First, the cipher design can be strengthened by carefully analyzing the propagation of bit-level differences and avoiding the existence of high-probability differential trails that can be exploited through PNDs. Incorporating stronger diffusion mechanisms, such as enhancing the round function complexity or using more balanced substitution permutation layers, can reduce the chances of bit differences propagating with exploitable bias.

Second, introducing round-wise permutation variations can help decorrelate the impact of nibble differences, thereby making it harder for the attacker to identify effective PND sets. Such design strategies increase unpredictability in difference propagation and make fault exploitation less feasible.

Finally, from an implementation perspective, techniques such as masking and controlled noise injection can be employed to conceal the effect of injected faults.

Therefore, a combination of design-level enhancements and implementation-level protections is essential to strengthen ARADI against PND-based differential fault attacks, while preserving its efficiency as a lightweight cipher.

4 Conclusion

In this paper, we analyzed the security of the ARADI cipher under differential fault attacks. By introducing a nibble-based fault model, we demonstrated that the master key can be recovered with a practical number of faults and reasonable computational effort. Our findings highlight that ARADI, despite its word-oriented design, remains vulnerable to carefully crafted bit-level fault attacks. This study emphasizes the need for stronger design strategies and countermeasures to improve the cipher’s resistance against fault-based cryptanalysis.

References

- [1] Subidh Ali, Debdeep Mukhopadhyay, and Michael Tunstall. Differential fault analysis of AES: towards reaching its limits. *J. Cryptogr. Eng.*, 3(2):73–97, 2013.
- [2] Nasour Bagheri, Reza Ebrahimpour, and Navid Ghaedi. New differential fault analysis on PRESENT. *EURASIP J. Adv. Signal Process.*, 2013:145, 2013.
- [3] Emanuele Bellini, Mattia Formenti, David G  rault, Juan Grados, Anna Hambitzer, Yun Ju Huang, Paul Huynh, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. Claasping ARADI: automated analysis of the ARADI block cipher. In Sourav Mukhopadhyay and Pantelimon Stanica, editors, *Progress in Cryptology - INDOCRYPT 2024 - 25th International Conference on Cryptology in India, Chennai, India, December 18-21, 2024, Proceedings, Part II*, volume 15496 of *Lecture Notes in Computer Science*, pages 90–113. Springer, 2024.
- [4] Emanuele Bellini, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. On the structural properties of toffoli gate composition in ARADI: implications for algebraic distinguishers. In Marc Fischlin and Veelasha Moonsamy, editors, *Applied*

- Cryptography and Network Security - 23rd International Conference, ACNS 2025, Munich, Germany, June 23-26, 2025, Proceedings, Part II*, volume 15826 of *Lecture Notes in Computer Science*, pages 400–425. Springer, 2025.
- [5] Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. In Burton S. Kaliski Jr., editor, *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*, volume 1294 of *Lecture Notes in Computer Science*, pages 513–525. Springer, 1997.
 - [6] Johannes Blömer and Volker Krummel. Fault based collision attacks on AES. In Luca Breveglieri, Israel Koren, David Naccache, and Jean-Pierre Seifert, editors, *Fault Diagnosis and Tolerance in Cryptography, Third International Workshop, FDTC 2006, Yokohama, Japan, October 10, 2006, Proceedings*, volume 4236 of *Lecture Notes in Computer Science*, pages 106–120. Springer, 2006.
 - [7] Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the importance of checking cryptographic protocols for faults (extended abstract). In Walter Fumy, editor, *Advances in Cryptology - EUROCRYPT '97, International Conference on the Theory and Application of Cryptographic Techniques, Konstanz, Germany, May 11-15, 1997, Proceeding*, volume 1233 of *Lecture Notes in Computer Science*, pages 37–51. Springer, 1997.
 - [8] Wei Cheng, Yongbin Zhou, and Laurent Sauvage. Differential fault analysis on midori. In Kwok-Yan Lam, Chi-Hung Chi, and Sihan Qing, editors, *Information and Communications Security - 18th International Conference, ICICS 2016, Singapore, November 29 - December 2, 2016, Proceedings*, volume 9977 of *Lecture Notes in Computer Science*, pages 307–317. Springer, 2016.
 - [9] Christoph Dobraunig, Maria Eichlseder, Thomas Korak, Stefan Mangard, Florian Mendel, and Robert Primas. Exploiting ineffective fault inductions on symmetric cryptography. *IACR Cryptol. ePrint Arch.*, page 71, 2018.
 - [10] Orr Dunkelman and Shibam Ghosh. Improved key-recovery attacks on aradi. *Cryptology ePrint Archive*, 2025.
 - [11] Sho Endo, Yang Li, Naofumi Homma, Kazuo Sakiyama, Kazuo Ohta, Daisuke Fujimoto, Makoto Nagata, Toshihiro Katashita, Jean-Luc Danger, and Takafumi Aoki. A silicon-level countermeasure against fault sensitivity analysis and its evaluation. *IEEE Trans. Very Large Scale Integr. Syst.*, 23(8):1429–1438, 2015.
 - [12] Patricia Greene, Mark Motley, and Bryan Weeks. ARADI and LLAMA: Low-latency cryptography for memory encryption. *Cryptology ePrint Archive*, Paper 2024/1240, 2024.
 - [13] Mustafa Khairallah and Trevor Yap. Attacking single-cycle ciphers on modern FPGAs featuring explainable deep learning. *Cryptology ePrint Archive*, Paper 2025/478, 2025.
 - [14] Anup Kumar Kundu, Aikata, Banashri Karmakar, and Dhiman Saha. Fault analysis of the PRINCE family of lightweight ciphers. *J. Cryptogr. Eng.*, 12(4):475–494, 2022.
 - [15] Donggeun Kwon and Seokhie Hong. Side-channel attack on aradi in non-profiling scenarios. *IEEE Access*, 13:122628–122635, 2025.
 - [16] Sandip Kumar Mondal, Prakash Dey, Himadry Sekhar Roy, Avishek Adhikari, and Subhamoy Maitra. Improved fault analysis on subterranean 2.0. *IEEE Transactions on Computers*, 73(06):1631–1639, jun 2024.

- [17] Dibyendu Roy, Bhagwan Bathe, and Subhamoy Maitra. Differential fault attack on kreyvium & flip. *IEEE Transactions on Computers*, 70(12):2161–2167, 2021.
- [18] Rajat Sadhukhan, Sayandeep Saha, Sudipta Paria, Swarup Bhunia, and Debdeep Mukhopadhyay. VALIANT: an EDA flow for side-channel leakage evaluation and tailored protection. *IEEE Trans. Computers*, 73(2):436–450, 2024.
- [19] Ling Song and Lei Hu. Differential fault attack on the PRINCE block cipher. In Gildas Avoine and Orhun Kara, editors, *Lightweight Cryptography for Security and Privacy - Second International Workshop, LightSec 2013, Gebze, Turkey, May 6-7, 2013, Revised Selected Papers*, volume 8162 of *Lecture Notes in Computer Science*, pages 43–54. Springer, 2013.
- [20] Mate Soos, Karsten Chu, and Christoph Wintersteiger. CryptoMiniSat – An Advanced Incremental SAT Solver. In *SAT*, pages 317–321, 2009.
- [21] Qingyuan Yu, Xiaoyang Dong, Lingyue Qin, Yongze Kang, Keting Jia, Xiaoyun Wang, and Guoyan Zhang. Automatic search of meet-in-the-middle differential fault analysis on aes-like ciphers. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(4):1–31, 2023.
- [22] Fan Zhang, Xiaoxuan Lou, Xinjie Zhao, Shivam Bhasin, Wei He, Ruyi Ding, Samiya Qureshi, and Kui Ren. Persistent fault analysis on block ciphers. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):150–172, 2018.

Table 3: Bit-based fault propagation for the fault injected in the word z

Fault In- jected bit	Active Bits	
	Beginning of 15th round	End of 16th round ($a \in \{w, x, y, z\}$)
$z^{14}[0]$	$z^{15}[0, 9, 30], y^{15}[0, 9, 30], w^{15}[0, 9, 30]$	$a^{16}[0, 1, 7, 8, 9, 16, 22, 23, 30]$
$z^{14}[1]$	$z^{15}[1, 10, 31], y^{15}[1, 10, 31], w^{15}[1, 10, 31]$	$a^{16}[1, 2, 8, 9, 10, 17, 23, 24, 31]$
$z^{14}[2]$	$z^{15}[2, 11, 16], y^{15}[2, 11, 16], w^{15}[2, 11, 16]$	$a^{16}[2, 3, 9, 10, 11, 16, 18, 24, 25]$
$z^{14}[3]$	$z^{15}[3, 12, 17], y^{15}[3, 12, 17], w^{15}[3, 12, 17]$	$a^{16}[3, 4, 10, 11, 12, 17, 19, 25, 26]$
$z^{14}[4]$	$z^{15}[4, 13, 18], y^{15}[4, 13, 18], w^{15}[4, 13, 18]$	$a^{16}[4, 5, 11, 12, 13, 18, 19, 26, 27]$
$z^{14}[5]$	$z^{15}[5, 14, 19], y^{15}[5, 14, 19], w^{15}[5, 14, 19]$	$a^{16}[5, 6, 12, 13, 14, 19, 21, 27, 28]$
$z^{14}[6]$	$z^{15}[6, 15, 20], y^{15}[6, 15, 20], w^{15}[6, 15, 20]$	$a^{16}[6, 7, 13, 14, 15, 20, 22, 28, 29]$
$z^{14}[7]$	$z^{15}[0, 7, 21], y^{15}[0, 7, 21], w^{15}[0, 7, 21]$	$a^{16}[0, 7, 8, 14, 15, 21, 23, 29, 30]$
$z^{14}[8]$	$z^{15}[1, 8, 22], y^{15}[1, 8, 22], w^{15}[1, 8, 22]$	$a^{16}[0, 1, 8, 9, 15, 22, 24, 30, 31]$
$z^{14}[9]$	$z^{15}[2, 9, 23], y^{15}[2, 9, 23], w^{15}[2, 9, 23]$	$a^{16}[0, 1, 2, 9, 10, 16, 23, 25, 31]$
$z^{14}[10]$	$z^{15}[3, 10, 24], y^{15}[3, 10, 24], w^{15}[3, 10, 24]$	$a^{16}[1, 2, 3, 10, 11, 16, 17, 24, 26]$
$z^{14}[11]$	$z^{15}[4, 11, 25], y^{15}[4, 11, 25], w^{15}[4, 11, 25]$	$a^{16}[2, 3, 4, 11, 12, 17, 18, 25, 27]$
$z^{14}[12]$	$z^{15}[5, 12, 26], y^{15}[5, 12, 26], w^{15}[5, 12, 26]$	$a^{16}[3, 4, 5, 12, 13, 18, 19, 26, 28]$
$z^{14}[13]$	$z^{15}[6, 13, 27], y^{15}[6, 13, 27], w^{15}[6, 13, 27]$	$a^{16}[4, 5, 6, 13, 14, 19, 20, 27, 29]$
$z^{14}[14]$	$z^{15}[7, 14, 28], y^{15}[6, 13, 27], w^{15}[6, 13, 27]$	$a^{16}[5, 6, 7, 14, 15, 20, 21, 28, 30]$
$z^{14}[15]$	$z^{15}[8, 15, 29], y^{15}[6, 13, 27], w^{15}[6, 13, 27]$	$a^{16}[0, 6, 7, 8, 15, 21, 22, 29, 31]$
$z^{14}[16]$	$z^{15}[4, 16, 25], y^{15}[4, 16, 25], w^{15}[4, 16, 25]$	$a^{16}[2, 4, 9, 12, 16, 17, 24, 25, 27]$
$z^{14}[17]$	$z^{15}[5, 17, 26], y^{15}[5, 17, 26], w^{15}[5, 17, 26]$	$a^{16}[3, 5, 10, 13, 17, 18, 25, 26, 28]$
$z^{14}[18]$	$z^{15}[6, 18, 27], y^{15}[6, 18, 27], w^{15}[6, 18, 27]$	$a^{16}[4, 6, 11, 14, 18, 19, 26, 27, 29]$
$z^{14}[19]$	$z^{15}[7, 19, 28], y^{15}[7, 19, 28], w^{15}[7, 19, 28]$	$a^{16}[5, 7, 12, 15, 19, 20, 27, 28, 30]$
$z^{14}[20]$	$z^{15}[8, 20, 29], y^{15}[8, 20, 29], w^{15}[8, 20, 29]$	$a^{16}[0, 6, 8, 13, 20, 21, 28, 29, 31]$
$z^{14}[21]$	$z^{15}[9, 21, 30], y^{15}[9, 21, 30], w^{15}[9, 21, 30]$	$a^{16}[1, 7, 9, 14, 16, 21, 22, 29, 30]$
$z^{14}[22]$	$z^{15}[10, 22, 31], y^{15}[10, 22, 31], w^{15}[10, 22, 31]$	$a^{16}[2, 8, 10, 15, 17, 22, 23, 30, 31]$
$z^{14}[23]$	$z^{15}[11, 16, 23], y^{15}[11, 16, 23], w^{15}[11, 16, 23]$	$a^{16}[0, 3, 9, 11, 16, 18, 23, 24, 31]$
$z^{14}[24]$	$z^{15}[12, 17, 24], y^{15}[12, 17, 24], w^{15}[12, 17, 24]$	$a^{16}[1, 4, 10, 12, 16, 17, 19, 24, 25]$
$z^{14}[25]$	$z^{15}[13, 18, 25], y^{15}[13, 18, 25], w^{15}[13, 18, 25]$	$a^{16}[2, 5, 11, 13, 17, 18, 20, 25, 26]$
$z^{14}[26]$	$z^{15}[14, 19, 26], y^{15}[14, 19, 26], w^{15}[14, 19, 26]$	$a^{16}[3, 6, 12, 14, 18, 19, 21, 26, 27]$
$z^{14}[27]$	$z^{15}[15, 20, 27], y^{15}[15, 20, 27], w^{15}[15, 20, 27]$	$a^{16}[4, 7, 13, 15, 19, 20, 22, 27, 28]$
$z^{14}[28]$	$z^{15}[0, 21, 28], y^{15}[0, 21, 28], w^{15}[0, 21, 28]$	$a^{16}[0, 5, 8, 14, 20, 21, 23, 28, 29]$
$z^{14}[29]$	$z^{15}[1, 22, 29], y^{15}[1, 22, 29], w^{15}[1, 22, 29]$	$a^{16}[1, 6, 9, 15, 21, 22, 24, 29, 30]$
$z^{14}[30]$	$z^{15}[2, 23, 30], y^{15}[2, 23, 30], w^{15}[2, 23, 30]$	$a^{16}[0, 2, 7, 10, 22, 23, 25, 30, 31]$
$z^{14}[31]$	$z^{15}[3, 24, 31], y^{15}[3, 24, 31], w^{15}[3, 24, 31]$	$a^{16}[1, 3, 8, 11, 16, 23, 24, 26, 31]$

Table 7: Nibble-based fault propagation pattern for the fault injected in the word z

Nibble Fault	Active Bits	
Beginning of 15th round	End of 15th round	End of 16th round ($a \in \{w, x, y, z\}$)
0x0000000f	$z^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31],$ $y^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31],$ $w^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31]$	[0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 16, 17, 18, 19, 22, 23, 24, 25, 26, 30, 31]
0x0000001e	$z^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31],$ $y^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31],$ $w^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31]$	[1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 16, 17, 18, 19, 20, 23, 24, 25, 26, 27, 31]
0x0000003c	$z^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19],$ $y^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19],$ $w^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19]$	[2, 3, 4, 5, 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 21, 24, 25, 26, 27, 28]
0x00000078	$z^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20],$ $y^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20],$ $w^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20]$	[3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 25, 26, 27, 28, 29]
0x000000f0	$z^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21],$ $y^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21],$ $w^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21]$	[0, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 30]
0x000001e0	$z^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22],$ $y^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22],$ $w^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22]$	[0, 1, 5, 6, 7, 8, 9, 12, 13, 14, 15, 19, 20, 21, 22, 23, 24, 27, 28, 29, 30, 31]
0x000003c0	$z^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23],$ $y^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23],$ $w^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23]$	[0, 1, 2, 6, 7, 8, 9, 10, 13, 14, 15, 16, 20, 21, 22, 23, 24, 25, 28, 29, 30, 31]
0x00000780	$z^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24],$ $y^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24],$ $w^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24]$	[0, 1, 2, 3, 7, 8, 9, 10, 11, 14, 15, 16, 17, 21, 22, 23, 24, 25, 26, 29, 30, 31]
0x00000f00	$z^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25],$ $y^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25],$ $w^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25]$	[0, 1, 2, 3, 4, 8, 9, 10, 11, 12, 15, 16, 17, 18, 22, 23, 24, 25, 26, 27, 30, 31]
0x00001e00	$z^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26],$ $y^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26],$ $w^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26]$	[0, 1, 2, 3, 4, 5, 9, 10, 11, 12, 13, 16, 17, 18, 19, 23, 24, 25, 26, 27, 28, 31]
0x00003c00	$z^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27],$ $y^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27],$ $w^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27]$	[1, 2, 3, 4, 5, 6, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 24, 25, 26, 27, 28, 29]
0x00007800	$z^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28],$ $y^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28],$ $w^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28]$	[2, 3, 4, 5, 6, 7, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 25, 26, 27, 28, 29, 30]
0x0000f000	$z^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29],$ $y^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29],$ $w^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29]$	[0, 3, 4, 5, 6, 7, 8, 12, 13, 14, 15, 18, 19, 20, 21, 22, 26, 27, 28, 29, 30, 31]
0x0001e000	$z^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29],$ $y^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29],$ $w^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29]$	[0, 2, 4, 5, 6, 7, 8, 9, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 24, 25, 27, 28, 29, 30, 31]
0x0003c000	$z^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29],$ $y^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29],$ $w^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29]$	[0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 24, 25, 26, 27, 28, 29, 30, 31]
0x00078000	$z^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29],$ $y^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29],$ $w^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29]$	[0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 24, 25, 26, 27, 28, 29, 31]
0x000f0000	$z^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28],$ $y^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28],$ $w^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28]$	[2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27, 28, 29, 30]
0x001e0000	$z^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29],$ $y^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29],$ $w^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29]$	[0, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 25, 26, 27, 28, 29, 30, 31]
0x003c0000	$z^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30],$ $y^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30],$ $w^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30]$	[0, 1, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 26, 27, 28, 29, 30, 31]
0x00780000	$z^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31],$ $y^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31],$ $w^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31]$	[0, 1, 2, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 27, 28, 29, 30, 31]
0x00f00000	$z^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31],$ $y^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31],$ $w^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31]$	[0, 1, 2, 3, 6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24, 28, 29, 30, 31]
0x01e00000	$z^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31],$ $y^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31],$ $w^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31]$	[0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 14, 15, 16, 17, 18, 19, 21, 22, 23, 24, 25, 29, 30, 31]
0x03c00000	$z^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31],$ $y^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31],$ $w^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31]$	[0, 1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 30, 31]
0x07800000	$z^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26],$ $y^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26],$ $w^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26]$	[0, 1, 2, 3, 4, 5, 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 31]
0x0f000000	$z^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27],$ $y^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27],$ $w^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27]$	[1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 26, 27, 28]
0x1e000000	$z^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28],$ $y^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28],$ $w^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28]$	[0, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23, 25, 26, 27, 28, 29]
0x3c000000	$z^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29],$ $y^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29],$ $w^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29]$	[0, 1, 3, 4, 5, 6, 7, 8, 9, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 24, 26, 27, 28, 29, 30]
0x78000000	$z^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30],$ $y^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30],$ $w^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30]$	[0, 1, 2, 4, 5, 6, 7, 8, 9, 10, 13, 14, 15, 19, 20, 21, 22, 23, 24, 25, 27, 28, 29, 30, 31]
0xf0000000	$z^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31],$ $y^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31],$ $w^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31]$	[0, 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 14, 15, 16, 20, 21, 22, 23, 24, 25, 26, 28, 29, 30, 31]

Table 8: Bit-based fault propagation for the fault injected in the word x

Fault Injected bit	Active Bits	
	Beginning of 15th round	End of 16th round ($a \in \{w, x, y, z\}$)
$x^{14}[0]$	$x^{15}[0, 9, 30], y^{15}[0, 9, 30], w^{15}[0, 9, 30], z^{15}[0, 9, 30]$	$a^{16}[0, 1, 7, 8, 9, 16, 22, 23, 30]$
$x^{14}[1]$	$x^{15}[1, 10, 31], y^{15}[1, 10, 31], w^{15}[1, 10, 31], z^{15}[1, 10, 31]$	$a^{16}[1, 2, 8, 9, 10, 17, 23, 24, 31]$
$x^{14}[2]$	$x^{15}[2, 11, 16], y^{15}[2, 11, 16], w^{15}[2, 11, 16], z^{15}[2, 11, 16]$	$a^{16}[2, 3, 9, 10, 11, 16, 18, 24, 25]$
$x^{14}[3]$	$x^{15}[3, 12, 17], y^{15}[3, 12, 17], w^{15}[3, 12, 17], z^{15}[3, 12, 17]$	$a^{16}[3, 4, 10, 11, 12, 17, 19, 25, 26]$
$x^{14}[4]$	$x^{15}[4, 13, 18], y^{15}[4, 13, 18], w^{15}[4, 13, 18], z^{15}[4, 13, 18]$	$a^{16}[4, 5, 11, 12, 13, 18, 19, 26, 27]$
$x^{14}[5]$	$x^{15}[5, 14, 19], y^{15}[5, 14, 19], w^{15}[5, 14, 19], z^{15}[5, 14, 19]$	$a^{16}[5, 6, 12, 13, 14, 19, 21, 27, 28]$
$x^{14}[6]$	$x^{15}[6, 15, 20], y^{15}[6, 15, 20], w^{15}[6, 15, 20], z^{15}[6, 15, 20]$	$a^{16}[6, 7, 13, 14, 15, 20, 22, 28, 29]$
$x^{14}[7]$	$x^{15}[0, 7, 21], y^{15}[0, 7, 21], w^{15}[0, 7, 21], z^{15}[0, 7, 21]$	$a^{16}[0, 7, 8, 14, 15, 21, 23, 29, 30]$
$x^{14}[8]$	$x^{15}[1, 8, 22], y^{15}[1, 8, 22], w^{15}[1, 8, 22], z^{15}[1, 8, 22]$	$a^{16}[0, 1, 8, 9, 15, 22, 24, 30, 31]$
$x^{14}[9]$	$x^{15}[2, 9, 23], y^{15}[2, 9, 23], w^{15}[2, 9, 23], z^{15}[2, 9, 23]$	$a^{16}[0, 1, 2, 9, 10, 16, 23, 25, 31]$
$x^{14}[10]$	$x^{15}[3, 10, 24], y^{15}[3, 10, 24], w^{15}[3, 10, 24], z^{15}[3, 10, 24]$	$a^{16}[1, 2, 3, 10, 11, 16, 17, 24, 26]$
$x^{14}[11]$	$x^{15}[4, 11, 25], y^{15}[4, 11, 25], w^{15}[4, 11, 25], z^{15}[4, 11, 25]$	$a^{16}[2, 3, 4, 11, 12, 17, 18, 25, 27]$
$x^{14}[12]$	$x^{15}[5, 12, 26], y^{15}[5, 12, 26], w^{15}[5, 12, 26], z^{15}[5, 12, 26]$	$a^{16}[3, 4, 5, 12, 13, 18, 19, 26, 28]$
$x^{14}[13]$	$x^{15}[6, 13, 27], y^{15}[6, 13, 27], w^{15}[6, 13, 27], z^{15}[6, 13, 27]$	$a^{16}[4, 5, 6, 13, 14, 19, 20, 27, 29]$
$x^{14}[14]$	$x^{15}[7, 14, 28], y^{15}[6, 13, 27], w^{15}[6, 13, 27], z^{15}[6, 13, 27]$	$a^{16}[5, 6, 7, 14, 15, 20, 21, 28, 30]$
$x^{14}[15]$	$x^{15}[8, 15, 29], y^{15}[6, 13, 27], w^{15}[6, 13, 27], z^{15}[6, 13, 27]$	$a^{16}[0, 6, 7, 8, 15, 21, 22, 29, 31]$
$x^{14}[16]$	$x^{15}[4, 16, 25], y^{15}[4, 16, 25], w^{15}[4, 16, 25], z^{15}[4, 16, 25]$	$a^{16}[2, 4, 9, 12, 16, 17, 24, 25, 27]$
$x^{14}[17]$	$x^{15}[5, 17, 26], y^{15}[5, 17, 26], w^{15}[5, 17, 26], z^{15}[5, 17, 26]$	$a^{16}[3, 5, 10, 13, 17, 18, 25, 26, 28]$
$x^{14}[18]$	$x^{15}[6, 18, 27], y^{15}[6, 18, 27], w^{15}[6, 18, 27], z^{15}[6, 18, 27]$	$a^{16}[4, 6, 11, 14, 18, 19, 26, 27, 29]$
$x^{14}[19]$	$x^{15}[7, 19, 28], y^{15}[7, 19, 28], w^{15}[7, 19, 28], z^{15}[7, 19, 28]$	$a^{16}[5, 7, 12, 15, 19, 20, 27, 28, 30]$
$x^{14}[20]$	$x^{15}[8, 20, 29], y^{15}[8, 20, 29], w^{15}[8, 20, 29], z^{15}[8, 20, 29]$	$a^{16}[0, 6, 8, 13, 20, 21, 28, 29, 31]$
$x^{14}[21]$	$x^{15}[9, 21, 30], y^{15}[9, 21, 30], w^{15}[9, 21, 30], z^{15}[9, 21, 30]$	$a^{16}[1, 7, 9, 14, 16, 21, 22, 29, 30]$
$x^{14}[22]$	$x^{15}[10, 22, 31], y^{15}[10, 22, 31], w^{15}[10, 22, 31], z^{15}[10, 22, 31]$	$a^{16}[2, 8, 10, 15, 17, 22, 23, 30, 31]$
$x^{14}[23]$	$x^{15}[11, 16, 23], y^{15}[11, 16, 23], w^{15}[11, 16, 23], y^{15}[11, 16, 23]$	$a^{16}[0, 3, 9, 11, 16, 18, 23, 24, 31]$
$x^{14}[24]$	$x^{15}[12, 17, 24], y^{15}[12, 17, 24], w^{15}[12, 17, 24], z^{15}[12, 17, 24]$	$a^{16}[1, 4, 10, 12, 16, 17, 19, 24, 25]$
$x^{14}[25]$	$x^{15}[13, 18, 25], y^{15}[13, 18, 25], w^{15}[13, 18, 25], z^{15}[13, 18, 25]$	$a^{16}[2, 5, 11, 13, 17, 18, 20, 25, 26]$
$x^{14}[26]$	$x^{15}[14, 19, 26], y^{15}[14, 19, 26], w^{15}[14, 19, 26], z^{15}[14, 19, 26]$	$a^{16}[3, 6, 12, 14, 18, 19, 21, 26, 27]$
$x^{14}[27]$	$x^{15}[15, 20, 27], y^{15}[15, 20, 27], w^{15}[15, 20, 27], z^{15}[15, 20, 27]$	$a^{16}[4, 7, 13, 15, 19, 20, 22, 27, 28]$
$x^{14}[28]$	$x^{15}[0, 21, 28], y^{15}[0, 21, 28], w^{15}[0, 21, 28], z^{15}[0, 21, 28]$	$a^{16}[0, 5, 8, 14, 20, 21, 23, 28, 29]$
$x^{14}[29]$	$x^{15}[1, 22, 29], y^{15}[1, 22, 29], w^{15}[1, 22, 29], z^{15}[1, 22, 29]$	$a^{16}[1, 6, 9, 15, 21, 22, 24, 29, 30]$
$x^{14}[30]$	$x^{15}[2, 23, 30], y^{15}[2, 23, 30], w^{15}[2, 23, 30], z^{15}[2, 23, 30]$	$a^{16}[0, 2, 7, 10, 22, 23, 25, 30, 31]$
$x^{14}[31]$	$x^{15}[3, 24, 31], y^{15}[3, 24, 31], w^{15}[3, 24, 31], z^{15}[3, 24, 31]$	$a^{16}[1, 3, 8, 11, 16, 23, 24, 26, 31]$

Table 9: Nibble-based fault propagation pattern for the fault injected in the word x

Nibble Fault	Active Bits	
Beginning of 15th round	End of 15th round	End of 16th round ($a \in \{w, x, y, z\}$)
0x0000000f	$w^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31],$ $y^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31],$ $z^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31]$	$x^{15}[0, 1, 2, 3, 9, 10, 11, 12, 16, 17, 30, 31],$ [0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 16, 17, 18, 19, 22, 23, 24, 25, 26, 30, 31]
0x0000001e	$w^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31],$ $y^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31],$ $z^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31]$	$x^{15}[1, 2, 3, 4, 10, 11, 12, 13, 16, 17, 18, 31],$ [1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 16, 17, 18, 19, 20, 23, 24, 25, 26, 27, 31]
0x0000003c	$w^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19],$ $y^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19],$ $z^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19]$	$x^{15}[2, 3, 4, 5, 11, 12, 13, 14, 16, 17, 18, 19],$ [2, 3, 4, 5, 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 21, 24, 25, 26, 27, 28]
0x00000078	$w^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20],$ $y^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20],$ $z^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20]$	$x^{15}[3, 4, 5, 6, 12, 13, 14, 15, 17, 18, 19, 20],$ [3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 25, 26, 27, 28, 29]
0x000000ff	$w^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21],$ $y^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21],$ $z^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21]$	$x^{15}[0, 4, 5, 6, 7, 13, 14, 15, 18, 19, 20, 21],$ [0, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 30]
0x000001e0	$w^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22],$ $y^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22],$ $z^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22]$	$x^{15}[0, 1, 5, 6, 7, 8, 14, 15, 19, 20, 21, 22],$ [0, 1, 5, 6, 7, 8, 9, 12, 13, 14, 15, 19, 20, 21, 22, 23, 24, 27, 28, 29, 30, 31]
0x000003c0	$w^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23],$ $y^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23],$ $z^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23]$	$x^{15}[0, 1, 2, 6, 7, 8, 9, 15, 20, 21, 22, 23],$ [0, 1, 2, 6, 7, 8, 9, 10, 13, 14, 15, 16, 20, 21, 22, 23, 24, 25, 28, 29, 30, 31]
0x00000780	$w^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24],$ $y^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24],$ $z^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24]$	$x^{15}[0, 1, 2, 3, 7, 8, 9, 10, 21, 22, 23, 24],$ [0, 1, 2, 3, 7, 8, 9, 10, 11, 14, 15, 16, 17, 21, 22, 23, 24, 25, 26, 29, 30, 31]
0x00000ff0	$w^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25],$ $y^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25],$ $z^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25]$	$x^{15}[1, 2, 3, 4, 8, 9, 10, 11, 22, 23, 24, 25],$ [0, 1, 2, 3, 4, 8, 9, 10, 11, 12, 15, 16, 17, 18, 22, 23, 24, 25, 26, 27, 30, 31]
0x00001e00	$w^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26],$ $y^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26],$ $z^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26]$	$x^{15}[2, 3, 4, 5, 9, 10, 11, 12, 23, 24, 25, 26],$ [0, 1, 2, 3, 4, 5, 9, 10, 11, 12, 13, 16, 17, 18, 19, 23, 24, 25, 26, 27, 28, 31]
0x00003c00	$w^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27],$ $y^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27],$ $z^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27]$	$x^{15}[3, 4, 5, 6, 10, 11, 12, 13, 24, 25, 26, 27],$ [1, 2, 3, 4, 5, 6, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 24, 25, 26, 27, 28, 29]
0x00007800	$w^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28],$ $y^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28],$ $z^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28]$	$x^{15}[4, 5, 6, 7, 11, 12, 13, 14, 25, 26, 27, 28],$ [2, 3, 4, 5, 6, 7, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 25, 26, 27, 28, 29, 30]
0x0000ff00	$w^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29],$ $y^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29],$ $z^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29]$	$x^{15}[5, 6, 7, 8, 12, 13, 14, 15, 26, 27, 28, 29],$ [0, 3, 4, 5, 6, 7, 8, 12, 13, 14, 15, 18, 19, 20, 21, 22, 26, 27, 28, 29, 30, 31]
0x0001e000	$w^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29],$ $y^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29],$ $z^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29]$	$x^{15}[4, 6, 7, 8, 13, 14, 15, 16, 25, 27, 28, 29],$ [0, 2, 4, 5, 6, 7, 8, 9, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 24, 25, 27, 28, 29, 30, 31]
0x0003c000	$w^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29],$ $y^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29],$ $z^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29]$	$x^{15}[4, 5, 7, 8, 14, 15, 16, 17, 25, 26, 28, 29],$ [0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 24, 25, 26, 27, 28, 29, 30, 31]
0x00078000	$w^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29],$ $y^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29],$ $z^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29]$	$x^{15}[4, 5, 6, 8, 15, 16, 17, 18, 25, 26, 27, 29],$ [0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 24, 25, 26, 27, 28, 29, 31]
0x000ff000	$w^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28],$ $y^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28],$ $z^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28]$	$x^{15}[4, 5, 6, 7, 16, 17, 18, 19, 25, 26, 27, 28],$ [2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 24, 25, 26, 27, 28, 29, 30]
0x001e0000	$w^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29],$ $y^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29],$ $z^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29]$	$x^{15}[5, 6, 7, 8, 17, 18, 19, 20, 26, 27, 28, 29],$ [0, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 25, 26, 27, 28, 29, 30, 31]
0x003c0000	$w^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30],$ $y^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30],$ $z^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30]$	$x^{15}[6, 7, 8, 9, 18, 19, 20, 21, 27, 28, 29, 30],$ [0, 1, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 26, 27, 28, 29, 30, 31]
0x00780000	$w^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31],$ $y^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31],$ $z^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31]$	$x^{15}[7, 8, 9, 10, 19, 20, 21, 22, 28, 29, 30, 31],$ [0, 1, 2, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 27, 28, 29, 30, 31]
0x00ff0000	$w^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31],$ $y^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31],$ $z^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31]$	$x^{15}[8, 9, 10, 11, 16, 20, 21, 22, 23, 29, 30, 31],$ [0, 1, 2, 3, 6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24, 28, 29, 30, 31]
0x01e00000	$w^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31],$ $y^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31],$ $z^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31]$	$x^{15}[9, 10, 11, 12, 16, 17, 21, 22, 23, 24, 30, 31],$ [0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 14, 15, 16, 17, 18, 19, 21, 22, 23, 24, 25, 29, 30, 31]
0x03c00000	$w^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31],$ $y^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31],$ $z^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31]$	$x^{15}[10, 11, 12, 13, 16, 17, 18, 22, 23, 24, 25, 31],$ [0, 1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 30, 31]
0x07800000	$w^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26],$ $y^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26],$ $z^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26]$	$x^{15}[11, 12, 13, 14, 16, 17, 18, 19, 23, 24, 25, 26],$ 0, 1, 2, 3, 4, 5, 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 31
0x0ff00000	$w^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27],$ $y^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27],$ $z^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27]$	$x^{15}[12, 13, 14, 15, 17, 18, 19, 20, 24, 25, 26, 27],$ [1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 26, 27, 28]
0x1e000000	$w^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28],$ $y^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28],$ $z^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28]$	$x^{15}[0, 13, 14, 15, 18, 19, 20, 21, 25, 26, 27, 28],$ [0, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23, 25, 26, 27, 28, 29]
0x3c000000	$w^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29],$ $y^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29],$ $z^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29]$	$x^{15}[0, 1, 14, 15, 19, 20, 21, 22, 26, 27, 28, 29],$ [0, 1, 3, 4, 5, 6, 7, 8, 9, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23, 24, 26, 27, 28, 29, 30]
0x78000000	$w^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30],$ $y^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30],$ $z^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30]$	$x^{15}[0, 1, 2, 15, 20, 21, 22, 23, 27, 28, 29, 30],$ [0, 1, 2, 4, 5, 6, 7, 8, 9, 10, 13, 14, 15, 19, 20, 21, 22, 23, 24, 25, 27, 28, 29, 30, 31]
0xff000000	$w^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31],$ $y^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31],$ $z^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31]$	$x^{15}[0, 1, 2, 3, 21, 22, 23, 24, 28, 29, 30, 31],$ [0, 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 14, 15, 16, 20, 21, 22, 23, 24, 25, 26, 28, 29, 30, 31]